Chapter 10. Input and Output: Reading, Writing, and Directory Tricks

10.0 Introduction

Most programs need to interact with the outside world, and one common way of doing so is by reading and writing files. Files are usually stored on some persistent medium such as a disk drive; and, for the most part, we shall happily ignore the differences between files stored on a hard disk (and all the operating system–dependent filesystem types), a USB drive or SD card, a DVD-ROM, and other storage devices. For us, they're just files. And, like most other languages and OSes, Java extends the reading-and-writing model to network (socket) communications, which we'll touch on in Chapters 14 and 15.

Java provides many classes for input and output, as summarized later in Figure 10-1. This chapter covers all the normal input/output operations such as opening/closing and reading/writing files. Files are assumed to reside on some kind of file store or permanent storage. Distributed filesystems such as Apache Hadoop Distributed File System (HDFS), Sun's Network File System (NFS, common on Unix and available for Windows), SMB (the Windows network filesystem, available for Unix via the open source Samba program), and Filesystem in User SpacE (FUSE; implementations available for most Unix/Linux/macOS systems) are assumed to work just like disk filesystems, except where noted.

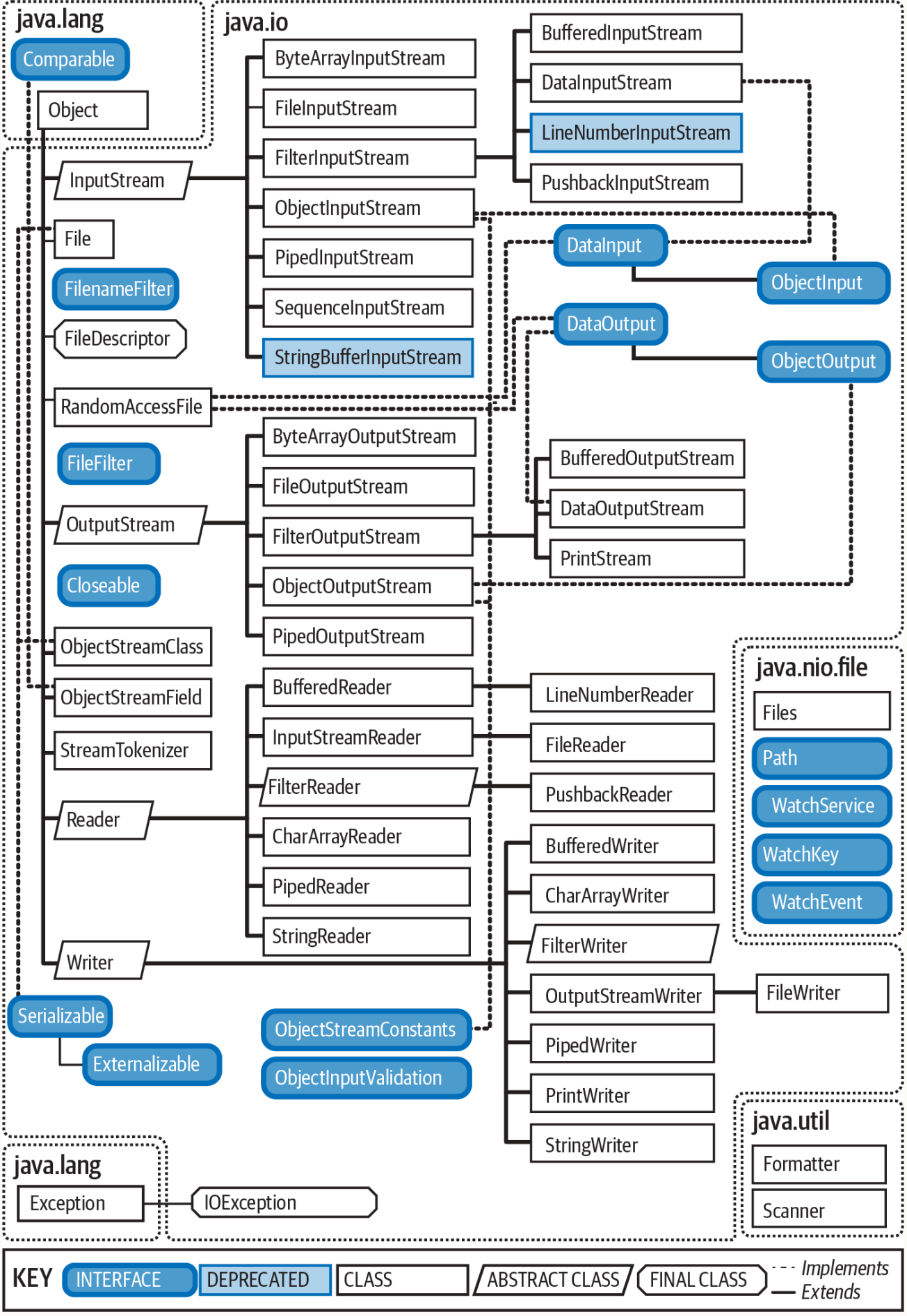


Figure 10-1. `java.io` classes for reading/writing

The support for reading and writing is in two major parts:

- The `InputStream` / `OutputStream` / `Reader` / `Writer` classes—the traditional methods of reading/writing files—have been largely unchanged since Java 1.0 and 1.1. Modern

Java provides a new class, `java.nio.file.Files`.

- All modern operating systems provide the means to organize groups of files into directories, or folders. This chapter covers directories: how to create them and how to navigate them. The `Files` class provides most of the support for processing directories, but it also introduces a number of convenience routines for easily reading, writing, and copying files, most of which are covered in this chapter. These are generally more convenient than using the traditional I/O classes. We cover both in this chapter.

The older `java.io.File` command is out of favor; best practice recommends using `Files` in its stead. Examples of the older `File` are left in the *javasrc* source repository for those who need them.

The first part of the chapter is largely devoted to the `Files` and `Path` classes in `java.nio.file`. These two classes provide the ability to list directories, obtain file status, rename and delete files on disk, create directories, and perform other filesystem operations. They also provide the ability to read a file line by line into a `Stream<String>`. These two classes together largely supplant the older `java.io.File` class. They were introduced in Java 7, so very little new code should use the older `File` class.

Note that many of the methods of this class attempt to modify the permanent file store, or disk filesystem, of the computer on which you run them. Naturally, you might not have permission to change certain files in certain ways. If you don't have permission to perform the attempted operation, as detected by the underlying operating system, Java will throw an instance of the checked exception `IOException`. This must be caught (or declared in the `throws` clause) in any code that calls any method that tries to change the filesystem, and some methods that try to access the filesystem's contents.

There are (at least!) two different uses of the term *stream*. The first is for a stream of bytes to be read or written, and it is covered in this chapter. The second meaning refers to the streams mechanism (`Stream` interface), which is used in modern Java to refer to a connection among cooperating methods, discussed in Recipe 9.3. I'll try to keep these meanings straight by only using `InputStream` and/or `OutputStream` for the former, and `Stream` for the latter.

To give you control over the format of data that you read and write, the `Formatter` and `Scanner` classes provide formatting and scanning operations. `Formatter` allows many formatting tasks to be performed either into a `String` or to almost any output destination. `Scanner` parses many kinds of objects, again either from a `String` or from almost any input source. These are fairly powerful; each is given its own recipe in this chapter.

10.1 Discovering Filesystem Paths

Problem

You need a `Path` object to work with the `Files` class.

Solution

Use the methods of `Path` to obtain an instance.

Discussion

A `Path` object represents a path into the filesystem, that is, a set of directories starting at the root of the filesystem, and possibly a file, like *C:\Users\user\Downloads* or */home/ian/Downloads*. The filesystem object does not need to exist on disk at the time you create a `Path` representing it. The `Files` class can tell you whether the filesystem object represented by a given `Path` exists, and:

- If not, can bring that `Path` into being as a file or as a directory, open it for writing, etc.
- If so, can read the file, change the file's attributes or contents, or even destroy the file.

`Path` objects are easily created with `Path.of(String name)`, which has several overloads.[1] `Path` is an interface whose implementation is provided by a provider class called `Filesystem`. `Path` has many methods, listed in Table 10-1.

Table 10-1. Public static methods in `java.nio.file.Path`

Access	Return type	Method
static	`Path`	`of(String, String...);`
static	Path	of(URI);
abstract	`FileSystem`	`getFileSystem();`
abstract	boolean	isAbsolute();
abstract	`Path`	`getRoot();`
abstract	Path	getFileName();
abstract	`Path`	`getParent();`
abstract	int	getNameCount();
abstract	`Path`	`getName(int);`
abstract	Path	subpath(int, int);
abstract	`boolean`	`startsWith(Path);`
default	boolean	startsWith(String);
abstract	`boolean`	`endsWith(Path);`
default	boolean	endsWith(String);
abstract	`Path`	`normalize();`
abstract	Path	resolve(Path);
default	`Path`	`resolve(String);`
default	Path	resolveSibling(Path);
default	`Path`	`resolveSibling(String);`
abstract	Path	relativize(Path);
abstract	`URI`	`toUri();`
abstract	Path	toAbsolutePath();
abstract	`Path`	`toRealPath(LinkOption...) throws IOException;`
default	File	toFile();

Access	Return type	Method
abstract	`WatchKey`	`register(WatchService, WatchEvent$Kind<?>[], WatchEvent$Modifier...) throws IOException;`
default	WatchKey	register(WatchService, WatchEvent$Kind<?>...) throws IOException;
default	`Iterator<Path>`	`iterator();`
abstract	int	compareTo(Path);
abstract	`boolean`	`equals(Object);`
abstract	int	hashCode();
abstract	`String`	`toString();`
default	int	compareTo(Object);

Some of these methods are demonstrated in Example 10-1. The `register(WatchService...)` methods are described in Recipe 10.19.

Example 10-1. main/src/main/java/dir_file/PathDemo.java

```
import java.nio.file.Path;

public class PathDemo {

  public static void main(String[] args) {
    var p = Path.of("/etc/hosts");
    display("Path", p);
    display("isAbsolute", p.isAbsolute());
    display("getRoot", p.getRoot());
    display("getParent", p.getParent());
    var q = Path.of("PathDemo.java");
    display("endsWith(.php)", q.endsWith(".php"));
    display("endsWith(.java)", q.endsWith(".java"));
    var s = Path.of("/usr/bin/../../home/ian/../../etc/hosts");
    display("Normalize long path", s = s.normalize());
    display("Equals after normalize", s.equals(p));
    var t = Path.of("Foo.java");
    display("absolutePath", t.toAbsolutePath());
  }
```

To work with actual files in a filesystem, `Path` objects are typically used in conjunction with `Files`, as described in Recipe 10.2.

10.2 Getting and Setting File and Directory Information: Files and Path

Problem

You need to know all you can about a given file on disk.

Solution

Use `java.nio.file.Files` methods.

Discussion

The `java.nio.file.Files` class has a plural name both to differentiate it from the legacy `File` class that it replaces and to remind us that it sometimes works on multiple files. There are no public constructors and no instance methods, only static methods in the `Files` class. These methods fall into two categories: informational and operational. The informational methods (see Table 10-2) simply give you information about one file, such as `boolean exists()` or `long size()`. The operational methods (see Table 10-3) either make changes to the filesystem or open a file for reading or writing. Each of the operational methods can throw the checked exception `IOException`; only a few of the informational methods can.

The vast majority of these methods have argument(s) of type `java.nio.file.Path`. A `Path` represents a path into the filesystem, and is described in Recipe 10.1. The `Files` class can tell you whether the file represented by a given `Path` exists, and:

- If not, can bring that `Path` into being as a file or as a directory, open it for writing, etc.
- If so, can read the file, change the file's attributes or contents, or even destroy the file.

`Files` in conjunction with `Path` offers pretty much everything you'd need to write a full-blown file manager application, let alone meet the needs of a more typical application

needing file information and/or directory access. The `Files` class has a series of static `boolean` methods that give basic information (see Table 10-2).

Table 10-2. Public static informational methods in `java.nio.file.Files`

Return type	Method	Notes
`boolean`	`exists(Path, LinkOption...);`	
`Object`	`getAttribute(Path, String, LinkOption...);`	
`<V extends FileAttributeView> V`	`getFileAttributeView(Path, Class<V>, LinkOption...);`	
`FileTime`	`getLastModifiedTime(Path, LinkOption...);`	
`UserPrincipal`	`getOwner(Path, LinkOption...);`	
`Set<PosixFilePermission>`	`getPosixFilePermissions(Path, LinkOption...);`	
`boolean`	`isDirectory(Path, LinkOption...);`	
`boolean`	`isExecutable(Path);`	If executable by current user
`boolean`	`isHidden(Path);`	If a “dot file” on Unix, or “hidden” attribute set on some OSes
`boolean`	`isReadable(Path);`	If readable by current user
`boolean`	`isRegularFile(Path, LinkOption...);`	
`boolean`	`isSameFile(Path, Path) throws IOException;`	Has to unwind filesys complexities like .. , symlinks, ...
`boolean`	`isSymbolicLink(Path);`	
`boolean`	`isWritable(Path);`	If writable by current user
`long`	`mismatch(Path, Path);`	
`boolean`	`notExists(Path, LinkOption...);`	
`String`	`probeContentType(Path) throws IOException;`	Tries to return MIME type of data

Return type	Method	Notes
Path	readSymbolicLink(Path) throws IOException;	
long	size(Path);	

By “current user” we mean the account under which the current JVM instance is being run.

Most of the methods in Table 10-2 are demonstrated in Example 10-2.

Example 10-2. main/src/main/java/io/FilesInfos.java

```
    println("exists", Files.exists(Path.of("/")));
    println("isDirectory", Files.isDirectory(Path.of("/")));
    println("isExecutable", Files.isExecutable(Path.of("/bin/cat")));
    println("isHidden", Files.isHidden(Path.of("~/.profile")));
    println("isReadable", Files.isReadable(Path.of("lines.txt")));
    println("isRegularFile", Files.isRegularFile(Path.of("lines.txt")));
    println("isSameFile", Files.isSameFile(Path.of("lines.txt"),
      Path.of("../main/lines.txt")));
    println("isSymbolicLink", Files.isSymbolicLink(Path.of("/var")));
    println("isWritable", Files.isWritable(Path.of("/tmp")));
    println("isDirectory", Files.isDirectory(Path.of("/")));
    println("notexists",
      Files.notExists(Path.of("no_such_file_as_skjfsjljwerjwj")));
    println("probeContentType", Files.probeContentType(Path.of("lines.txt")));
    println("readSymbolicLink", Files.readSymbolicLink(Path.of("/var")));
    println("size", Files.size(Path.of("lines.txt")));
```

Obviously the paths chosen are somewhat system-specific, but when run on my Unix system, the `boolean` methods all returned `true`, and the last three returned this:

```
probeContentType returned text/plain
readSymbolicLink returned private/var
size returned 78
```

Table 10-3 shows the methods that read and/or make changes to filesystem entities.

Table 10-3. Public static operational methods in `java.nio.file.Files`

Return type	Method
`long`	`copy(InputStream, Path, CopyOption...);`
long	copy(Path, OutputStream);
`Path`	`copy(Path, Path, CopyOption...);`
Path	createDirectories(Path, FileAttribute<?>...);
`Path`	`createDirectory(Path, FileAttribute<?>...);`
Path	createFile(Path, FileAttribute<?>...);
`Path`	`createLink(Path, Path);`
Path	createSymbolicLink(Path, Path, FileAttribute<?>...);
`Path`	`createTempDirectory(Path, String, FileAttribute<?>...);`
Path	createTempDirectory(String, FileAttribute<?>...);
`Path`	`createTempFile(Path, String, String, FileAttribute<?>...);`
Path	createTempFile(String, String, FileAttribute<?>...);
`void`	`delete(Path);`
boolean	deleteIfExists(Path);
`Stream<Path>`	`find(Path, int, BiPredicate<Path, BasicFileAttributes>, FileVisitOption...);`
Stream<String>	lines(Path);
`Stream<String>`	`lines(Path, Charset);`
Stream<Path>	list(Path);
`Path`	`move(Path, Path, CopyOption...);`
BufferedReader	newBufferedReader(Path);
`BufferedReader`	`newBufferedReader(Path, Charset);`
BufferedWriter	newBufferedWriter(Path, Charset, OpenOption...);
`BufferedWriter`	`newBufferedWriter(Path, OpenOption...);`
SeekableByteChannel	newByteChannel(Path, OpenOption...);

Return type	Method
SeekableByteChannel	newByteChannel(Path, Set<? extends OpenOption>, FileAttribute<?>...);
DirectoryStream<Path>	newDirectoryStream(Path);
DirectoryStream<Path>	newDirectoryStream(Path, String);
InputStream	newInputStream(Path, OpenOption...);
OutputStream	newOutputStream(Path, OpenOption...);
byte[]	readAllBytes(Path);
List<String>	readAllLines(Path);
List<String>	readAllLines(Path, Charset);
< A extends BasicFileAttributes> A	readAttributes(Path, Class<A>, LinkOption...);
Map<String, Object>	readAttributes(Path, String, LinkOption...);
String	readString(Path);
String	readString(Path, Charset);
Path	setAttribute(Path, String, Object, LinkOption...);
Path	setLastModifiedTime(Path, FileTime);
Path	setOwner(Path, UserPrincipal);
Path	setPosixFilePermissions(Path, Set<PosixFilePermission>);
Path	write(Path, Iterable<? extends CharSequence>, Charset, OpenOption...);
Path	write(Path, Iterable<? extends CharSequence>, OpenOption...);
Path	write(Path, byte[], OpenOption...);
Path	writeString(Path, CharSequence, Charset, OpenOption...);
Path	writeString(Path, CharSequence, OpenOption...);

To find the information about one file, you can use the informational methods in `Files` and `Path`, as shown in Example 10-3.

Example 10-3. main/src/main/java/dir_file/FileStatus.java (getting file information)

```
public class FileStatus {
 public static void main(String[] argv) throws IOException {

  // Ensure that a filename (or something) was given in argv[0]
  if (argv.length == 0) {
   System.err.println("Usage: FileStatus filename");
   System.exit(1);
  }
  for (String a : argv) {
   status(a);
  }
 }

 public static void status(String fileName) throws IOException {
  System.out.println("---" + fileName + "---");

  // Construct a Path object for the given file.
  Path p = Path.of(fileName);

  // See if it actually exists
  if (!Files.exists(p)) {
   System.out.println("file not found");
   System.out.println();  // Blank line
   return;
  }
  // Print full name
  System.out.println("Canonical name " + p.normalize());
  // Print parent directory if possible
  Path parent = p.getParent();
  if (parent != null) {
   System.out.println("Parent directory: " + parent);
  }
  // Check if the file is readable
  if (Files.isReadable(p)) {
   System.out.println(fileName + " is readable.");
  }
  // Check if the file is writable
  if (Files.isWritable(p)) {
   System.out.println(fileName + " is writable.");
  }
```

```
    // See if file, directory, or other. If file, print size.
    if (Files.isRegularFile(p)) {
      // Report on the file's size and possibly its type
      System.out.printf("File size is %d bytes, content type %s\n",
          Files.size(p),
          Files.probeContentType(p));
    } else if (Files.isDirectory(p)) {
      System.out.println("It's a directory");
    } else {
      System.out.println("I dunno! Neither a file nor a directory!");
    }

    // Report on the modification time.
    final FileTime d = Files.getLastModifiedTime(p);
    System.out.println("Last modified " + d);

    System.out.println();  // blank line between entries
  }
```

When run on Windows with the three arguments shown, it produces this output:

```
C:\javasrc\dir_file>java dir_file.FileStatus   / /tmp/id /autoexec.bat
---/---
Canonical name C:\
File is readable.
File is writable.
Last modified 1970-01-01T00:00:00.00000Z
It's a directory

---/tmp/id---
file not found

---/autoexec.bat---
Canonical name C:\AUTOEXEC.BAT
Parent directory: \
File is readable.
File is writable.
Last modified 2025-10-13T12:43:05.123918Z
File size is 308 bytes.
```

As you can see, the so-called *canonical name* not only includes a leading directory root of *C:*, but also has had the name converted to uppercase. You can tell I ran that on

Windows. That version of Windows did not maintain timestamps on directories; the value `0L` gets interpreted as January 1, 1970 (not coincidentally the same time base Unix has used since that time). On Unix, it behaves differently:

```
$ java dir_file.FileStatus / /tmp/id /autoexec.bat
---/---
Canonical name /
File is readable.
It's a directory
Last modified 2025-12-16T01:14:05.226108Z

---/tmp/id---
Canonical name /tmp/id
Parent directory: /tmp
File is readable.
File is writable.
File size is 36768 bytes, content type null
Last modified 2025-12-21T18:46:27.402108Z

---/autoexec.bat---
file not found

$
```

A typical Unix system has no *autoexec.bat* file. And Unix filenames (like those on a Mac) can consist of upper- and lowercase characters: what you type is what you get.

To list the filesystem entries named in a directory, use the `java.nio.file.Files` static method `Stream<Path> list(Path dir)`, passing the `Path` representing the directory. The `java.nio.file.Files` class contains several methods for working with directories. If you just want to list the contents of a directory, use its `list(Path)` method. For example, to list the filesystem entities named in the current directory, just write the following:

```
Files.list(Path.of(".")).forEach(System.out::println);
```

This can become a complete program known as a classless main (see Recipe 1.2), or be used in any release with the following code. Note that on many systems the `Path` objects are returned in the order they occur in the directory, which isn't sorted. In this simple example we use the `Stream.sorted()` method to order the entries alphabetically:

```
public class Ls {
  public static void main(String args[]) throws IOException {
    Files.list(Path.of("."))
      .sorted()
      .forEach(System.out::println);
  }
}
```

Of course, there's lots of room for elaboration. You could print the names in multiple columns across the page. Or you could even print down the page because you know the number of items in the list before you print. You could omit filenames with leading periods, as done by the Unix *ls* program. Or you could print the directory names first; I once used a directory lister called *lc* that did this, and I found it quite useful.

If you want to process the directory recursively, you should *not* check each entry to see if it's a file or directory and recurse on directories. Instead, you should use one of the `walk()` or `walkFileTree()` methods discussed in Recipe 10.20; these handle recursion for you. It may be more efficient to use one of the `find()` methods, rather than `walk()` with `filter()`. Example 10-4 shows the `find()` method being used to print all the book source files (names ending in *.asciidoc* or *.adoc*); the callback method is called with the `Path` object and a file attribute descriptor.

Example 10-4. main/src/main/java/dir_file/FindStream.java

```
void main() {
  var dirName = "/home/ian/book";
  try (Stream<Path> paths = Files.find(Path.of(dirName),
    Integer.MAX_VALUE,
    (path, attr) -> {
      return attr.isRegularFile() && (
        path.toString().endsWith(".adoc") ||
        path.toString().endsWith(".asciidoc"));
    })) {
    paths.forEach(System.out::println);
    }
  catch (IOException ex) {
    ex.printStackTrace();
  }
}
```

There is also a set of `Files.newDirectoryStream()` methods, with and without filter callbacks and other arguments, that return a `DirectoryStream<Path>`.

Legacy compatibility File—Files

To use a `Path` with legacy code that needs the older `java.io.File`, simply use `File oldType = Path.toFile()`:

```
jshell> Path p = Path.of("/");
p ==> /

jshell> File f = p.toFile();
f ==> /
```

To go the other way, the `File` class has been retrofitted with a `toPath()` method:

```
jshell> File f = new File("/");
f ==> /

jshell> Path p = f.toPath();
p ==> /
```

UNDERSTANDING I/O OPTIONS: STANDARDOPENOPTIONS, FILEATTRIBUTE, POSIXFILEATTRIBUTE, AND MORE

There are several sets of options that can be applied when creating or opening a file, particularly when using the `Files` class. The main option sets that can be applied include those listed in Table 10-4.

Table 10-4. Sets of options

Name	Examples	Usage/notes
`CopyOption`	`StandardCopyOption.REPLACE_EXISTING`	`Files` methods that copy
LinkOption	LinkOption. NOFOLLOW_LINKS	`Files` methods that write data or read attributes
`FileAttribute`		Name-value pair, used in `Files.create*()` methods
OpenOption	StandardOpenOption.READ,APPEND	Files.new{In,Out}putStream()
`PosixFilePermission`	`OWNER_READ,OTHER_WRITE`	`FileAttribute`
PosixFile Permissions	Set<PosixFilePermission>	Conversions to/from `rwx` strings

A list of the standard `OpenOption` values is in Table 10-5. These control how a file is to be accessed.

Table 10-5. `OpenOption StandardOpenOption` values

Name	Meaning
`APPEND`	Write at the end of an existing file instead of overwriting it.
`CREATE`	Create the file if it does not exist.
`CREATE_NEW`	Create the file only if it is new; fails with `FileAlreadyExistsException` if file already exists.
`DELETE_ON_CLOSE`	Delete the file when the stream is closed. Useful for temporary files.
`DSYNC`	Write data synchronously, i.e., every write is to be synchronized to disk immediately.
`READ`	Open the file for reading.
`SPARSE`	Create as a sparse file, e.g., for random-access writing.
`SYNC`	Write data and metadata synchronously, i.e., every write or attribute change is to be synchronized to disk immediately.
`TRUNCATE_EXISTING`	If the file exists, open for writing at the beginning, removing all contents at open time.
`WRITE`	Open for write access.

POSIX is the IEEE's[2] *P*ortable *O*perating *S*ystem Specification for Un*ix*-like operating systems. Java's `PosixPermission` and its wrapper `PosixPermissions` are used to control who can do what to a file on disk. These are based on the Unix/POSIX permissions laid down in early Unix systems in the early 1970s. There are three actors: owner (which Unix calls user), group, and other (everyone else). Groups is a Unix/POSIX mechanism: a user can be in one or many groups and has permissions based on all the groups they are in; this is an early form of privilege separation.

There are three permissions: read, write, and execute. The latter grants permission to execute a file, but is also used to grant permission to search (list) a directory. For decades these have been expressed as a nine-character permissions string. For example, `rwxr—r--` means the user has read, write, and execute permissions on a given file; other members of the file owner's group have read-only access, and everyone else also has read-only access. The `PosixPermissions` wrapper class has methods for converting between these concise strings and a `Set` of individual `PosixPermission` enum constants. The enum constants are the nine combinations of `OWNER`, `GROUP`, and `OTHERS` with

`READ`, `WRITE`, and `EXECUTE`. Here is a JShell example showing these file permission conversion routines:

```
jshell> Set<PosixFilePermission> perms =
  PosixFilePermissions.fromString("rwxr-xr--");
perms ==> [OWNER_READ, OWNER_WRITE, OWNER_EXECUTE, GROUP_READ,
  GROUP_EXECUTE, OTHERS_READ]

jshell> Set<PosixFilePermission> nPerms =
  Set.of(PosixFilePermission.OWNER_READ, PosixFilePermission.GROUP_READ);
nPerms ==> [GROUP_READ, OWNER_READ]

jshell> PosixFilePermissions.toString(nPerms)
$7 ==> "r--r-----"
```

You can further convert the `Set<PosixFilePermission>` into a `FileAttribute` to be used with the `Files` class `createFile()` or `createDirectory()` operations:

```
jshell> PosixFilePermissions.asFileAttribute(nPerms)
$8 ==> java.nio.file.attribute.PosixFilePermissions$1@ed17bee

jshell> $8.name()
$9 ==> "posix:permissions"

jshell> $8.value()
$10 ==> [OWNER_READ, GROUP_READ]

jshell> Files.createFile(Path.of("/tmp/xx"), $8);
$41 ==> /tmp/xx

jshell> /exit

$ ls -l /tmp/xx
-r--r-----  1 ian  wheel  0 Dec 23 11:14 /tmp/xx
$
```

We see the file was created with only owner-read and group-read permissions, as requested. Note that on *nix systems there is a user setting `umask` that may remove or mask out permissions, so what you ask for may not be exactly what you get.

You can examine the attributes of a file using the `FileAttribute` interface or its filesystem-specific subtypes. Here we'll use the `PosixFileAttributeView` to show the owner

and permissions of the file we created:

```
PosixFileAttributes attrs =
  Files.getFileAttributeView(filePath,
    PosixFileAttributeView.class)
  .readAttributes();
System.out.format("File %s Owned by %s has perms %s%n",
  filePath,
  attrs.owner().getName(),
  PosixFilePermissions.toString(attrs.permissions()));
```

There are other filesystem-specific views, such as `DosFileAttributeView` for use on FAT (file allocation table) filesystems. FAT was copied from CPM-86 into the earliest releases of MS-DOS, and expanded versions of it are still in use on USB memory cards and in consumer devices.

See Also

There is official documentation for `File`s and `Path`.

10.3 Creating and Deleting Files or Directories

Problem

You need to create a new file on disk but not immediately write any data into it. You need to create a directory before you can create files in it. Or you may need to delete a file or directory.

Solution

To create a file, use `Files.createFile()`. To create a directory, use `Files.createDirectory()` or `Files.createDirectories()`. To delete, use the `Files` object's `delete(Path)` or `deleteIfExists(Path)` method. These methods delete the files referred to by the `Path` argument (subject, of course, to permissions) and directories (subject to permissions and to the directory being empty).

Discussion

For an empty file, use a `java.nio.file.Files` object's `createFile(Path)` method. Use the `Files` class's `createDirectory()` or `createDirectories()` method to create a directory.

Files

You could easily create a new file by constructing a `FileOutputStream` or `FileWriter` (see Recipe 14.6). But then you'd have to remember to close it as well. Sometimes you want a file to exist, but you don't want to bother putting anything into it. This might be used, for example, as a simple form of interprogram communication: one program could test for the presence of a file and interpret that to mean that the other program has reached a certain state. An alternative simple semaphore is to use the `CREATE_NEW` option, which will fail if the file exists. Example 10-5 is code that simply creates an empty file for each name you give.

Example 10-5. main/src/main/java/dir_file/Creat.java (creation of a file on disk)

```
/** Create file(s) by name. Final "e" omitted in homage to UNIX system call. */
public class Creat {
  public static void main(String[] argv) throws IOException {

    // Ensure that a filename (or something) was given in argv[0]
    if (argv.length == 0) {
      throw new IllegalArgumentException("Usage: Creat filename [...]");
    }

    for (String arg : argv) {
      // Constructing a Path object doesn't affect the disk, but
      // the Files.createFile() method does.
      final Path p = Path.of(arg);
      final Path created = Files.createFile(p);
      System.out.println(created);
    }
  }
}
```

`java.nio.file.Files.createFile()` has an overload that takes a second argument of type `OpenOption`. This is an empty interface that is implemented by the `StandardOpenOption` enumeration. These options are listed in Table 10-5.

Directories

Of the two methods used for creating directories, `createDirectory()` creates just one directory, whereas `createDirectories()` creates any intermediate directories that are needed. For Unix/Linux folks, this is the difference between `mkdir` and `mkdir -p`. For example, if */home/ian* exists and is a directory, the call:

```
shell> Files.createDirectory(Path.of("/Users/ian/abc"))
$11 ==> /Users/ian/abc
```

will succeed (unless the directory is already there), but the call:

```
jshell> Files.createDirectory(Path.of("/Users/ian/once/twice/again"))
```

will fail with a `java.nio.file.NoSuchFileException` because the directory named *once* does not exist. To create this path of directories, as you might expect by now, use the plurally named `createDirectories()`:

```
jshell> Files.createDirectories(Path.of("/Users/ian/once/twice/again"))
$14 ==> /Users/ian/once/twice/again
```

Both variants return a `Path` object referring to the new directory if they succeed, and throw an exception if they fail. Notice that it is possible (but not probable) for `createDirectories()` to create some of the directories and then fail; in this case, the newly created directories are left in the filesystem.

Deletion is not complicated. Simply construct a `Path` object for the file you wish to delete, and call the static `Files.delete()` method:

```
public class Delete {
  public static void main(String[] argv) throws IOException {

    String fileToDelete = "Delete.java~";
    System.out.println("Deleting " + fileToDelete);
    // Construct a File object for the backup created by editing
    // this source file. The file probably already exists.
    // Some text editors create backups by putting ~ at end of filename.
    Path bkup = Path.of(fileToDelete);
    // Now, delete it:
```

```
    if (Files.exists(bkup)) {
      Files.delete(bkup);
      System.out.println("Done");
    } else {
      System.out.println("File not found");
    }
  }
}
```

Recall the caveat about permissions in the introduction to this chapter: if you don't have permission, you can get a return value of false or, possibly, an `Exception`. Note also that there are some differences between platforms. Some versions of Windows allow Java to remove a read-only file, but Unix does not allow you to remove a file unless you have write permission on the directory it's in. Nor does Unix allow you to remove a directory that isn't empty (there is even an exception, `DirectoryNotEmptyException`, for the latter case). Here is a version of `Delete` with reporting of success or failure:

```
public class Delete2 {

  static boolean hard = false; // True for delete, false for deleteIfExists

  public static void main(String[] argv) {
    for (String arg : argv) {
      if ("-h".equals(arg)) {
        hard = true;
        continue;
      }
      delete(arg);
    }
  }

  public static void delete(String fileName) {
    // Construct a File object for the file to be deleted.
    final Path target = Path.of(fileName);

    // Now, delete it:
    if (hard) {
      try {
        System.out.print("Using Files.delete(): ");
        Files.delete(target);
        System.err.println("** Deleted " + fileName + " **");
      } catch (IOException e) {
        System.out.println("Deleting " + fileName + " threw " + e);
      }
```

```
    } else {
      try {
        System.out.print("Using deleteIfExists(): ");
        if (Files.deleteIfExists(target)) {
          System.out.println("** Deleted " + fileName + " **");
        } else {
          System.out.println(
            "Deleting " + fileName + " returned false.");
        }
      } catch (IOException e) {
        System.out.println("Deleting " + fileName + " threw " + e);
      }
    }
  }
}
```

The `-h` option allows this program to switch between `delete()` and `deleteIfExists()`; you can see the difference by running it on things that exist, don't exist, and are not empty, using both methods. The output looks something like this on my Unix box:

```
$ ls -ld ?
-rw-r--r--  1 ian  512   0 Dec 21 16:35 a
drwxr-xr-x  2 ian  512  64 Dec 21 16:35 b
drwxr-xr-x  3 ian  512  96 Dec 21 16:22 c
$ java -cp target/classes dir_file.Delete2 a b c d
Using deleteIfExists(): ** Deleted a **
Using deleteIfExists(): ** Deleted b **
Using deleteIfExists(): Deleting c threw
 java.nio.file.DirectoryNotEmptyException: c
Using deleteIfExists(): Deleting d returned false.

# Here I put the files back the way they were, then run again with -h
$ java -cp target/classes dir_file.Delete2 -h a b c d
Using Files.delete(): ** Deleted a **
Using Files.delete(): ** Deleted b **
Using Files.delete(): Deleting c threw
 java.nio.file.DirectoryNotEmptyException: c
Using Files.delete(): Deleting d threw java.nio.file.NoSuchFileException: d
$ ls -l c
total 2
drwxr-xr-x  2 ian  ian  512 Oct  8 16:50 d
$ java dir_file.Delete2 c/d c
Using deleteIfExists(): ** Deleted c/d **
```

```
Using deleteIfExists(): ** Deleted c **
$
```

10.4 Changing a File's Name or Other Attributes

Problem

You need to change a file's name on disk or some of its other attributes, such as setting the file to read-only or changing its modification time.

Solution

To change the name (or location), use a `java.nio.file.Files` static `move()` method. For other attributes, use `setLastModifiedTime()` to change the timestamp, or use one of several other setters for mode or permission attributes.

Discussion

Similar to the Unix command line, there is no separate rename operation; the move methods provide all functions for putting a file somewhere else, whether that is under the same name in a different directory, a different name in the same directory, or a different name on a different disk or filesystem. Accordingly, the `Files.move()` method requires two `Path` objects, one referring to the existing file and another referring to the new name. Then call the `Files.move()` method, passing both `Path` objects, first the existing file and then the desired name. This is easier to see than to explain, so here goes:

```
public class Rename {
  public static void main(String[] argv) throws IOException {

    // Construct the Path object. Does NOT create a file on disk!
    final Path p = Path.of("MyCoolDocument"); // The file we will rename

    // Setup for the demo: create a new "old" file
    final Path oldName = Files.exists(p) ? p : Files.createFile(p);

    // Rename the file to "mydoc.bak"
    // Renaming needs a Path object for the target.
    final Path newName = Path.of("Mydoc.bak");
```

```
    Files.deleteIfExists(newName); // In case previous run left it there
    Path p2 = Files.move(oldName, newName);
    System.out.println(p + " renamed to " + p2);

    // What if a directory exists and you try to move a file into it?
    Path target = Path.of("temp");
    if (!Files.isDirectory(target)) {
      Files.createDirectory(target);
    }
    try {
      Files.move(p2, target);
    } catch (java.nio.file.FileAlreadyExistsException ex) {
      System.out.println("Caught expected FileAlreadyExistsException");
    }
    // Cleanup time!
    Files.delete(p2);
    Files.delete(target);
  }
}
```

For changing the attributes, there are several methods available, listed in Table 10-6. Each of these has a return value of type `boolean`, with `true` meaning success.

Table 10-6. Legacy file attribute setters

Method signature	Description
`setExecutable(boolean executable)`	Convenience method to set owner's execute permission for this file
`setExecutable(boolean executable, boolean ownerOnly)`	Sets the owner's or everybody's execute permission for this file
`setLastModified(long time)`	Sets the last-modified time of the file or directory that this file names
`setReadable(boolean readable)`	Convenience method to set owner's read permission for this file
`setReadable(boolean readable, boolean ownerOnly)`	Sets the owner's or everybody's read permission for this file
`setReadOnly()`	Convenience method for `setReadable(false)`
`setWritable(boolean writable)`	A convenience method to set the owner's write permission for this file
`setWritable(boolean writable, boolean ownerOnly)`	Sets owner's or everybody's write permission for this file

For the methods that take two arguments, the first enables or disables the feature on the given file that matches the method name, and the second controls whether the operation applies to the owner only or to everyone. The second argument is ignored if the file lives on a filesystem that doesn't support multiuser permissions or if the operating system doesn't support that. All the methods described in this recipe return `true` if they succeed and `false` otherwise.

For example, `boolean setReadable(boolean readable, boolean ownerOnly)` lets you specify who can read the given file. The `readable` argument is true or false depending on whether you want it to be readable or not. A value of `false` for the `ownerOnly` argument tries to extend the readability choice to all users on a multiuser operating system, and is ignored if not applicable.

`setLastModified()` allows you to play games with the modification time of a file. This is normally not a good game to play, but it is useful in some types of backup/restore programs. This method takes an argument that is the number of milliseconds (not seconds) since the beginning of Unix time (January 1, 1970). You can get the original value for the file by calling `getLastModified()` (see Recipe 10.2), or you can get the value for a given

date by calling the `ZonedDateTime`'s `toInstant().getEpochSecond()` method (see Recipe 6.3) and multiplying by 1,000 to convert seconds to milliseconds.

I encourage you to explore the operation of these methods using JShell (see Recipe 1.5). I'd suggest having a second window in which you can run `ls -l` or `dir` commands to see how the file is affected. Example 10-6 shows some of these methods on the legacy `File` class being explored in JShell.

Example 10-6. Exploring files

```
jshell> var f = File.createTempFile("foo", "bar");
f ==> /tmp/foo939130078908778098 4bar

jshell> f.createNewFile();
$4 ==> false

jshell> f.setReadOnly();
$5 ==> true

jshell> f.canRead();
$6 ==> true

jshell> f.canWrite();
$7 ==> false

jshell> f.setReadable(true);
$8 ==> true

jshell> f.canWrite();
$9 ==> false

jshell> f.setReadable(false, false);
$10 ==> true

jshell> f.canWrite();
$11 ==> false
```

10.5 About InputStreams/OutputStreams and

Readers/Writers

Problem

You need to read or write a file with text or binary data.

Solution

Use a `Reader` or `Writer` for text, or an `InputStream` or `OutputStream` for binary data.

Discussion

Java provides two sets of classes for reading and writing. The `InputStream` / `OutputStream` section of package java.io (see Figure 10-1) is for reading or writing bytes of data. Older languages tended to assume that a byte (which is a machine-specific collection of bits, usually eight bits on modern computers) is exactly the same thing as a character—a letter, digit, or other linguistic element. However, Java is designed to be used internationally, and eight bits is simply not enough to handle the many different character sets used around the world. Script-based languages, and pictographic languages like Chinese and Japanese, each have many more than 256 characters, the maximum that can be represented in an 8-bit byte. The unification of these many character code sets is called, not surprisingly, Unicode. Both Java and XML use Unicode as their character sets, allowing you to read and write text in any of these human languages. You should use `Reader`s and `Writer`s, not `Stream`s, for textual data.

Unicode itself doesn't solve the entire problem. Many of these human languages were used on computers long before Unicode was invented, and they didn't all pick the same representation as Unicode. And they all have zillions of files encoded in a particular representation that isn't Unicode. So routines are needed when reading and writing to convert between Unicode `String` objects used inside the Java machine and the particular external representation in which a user's files are written. These converters are packaged inside a powerful set of classes called `Reader`s and `Writer`s. `Reader`s and `Writer`s should always be used instead of `InputStream`s and `OutputStream`s when you want to deal with characters instead of bytes. We'll see more on this conversion, and how to specify which conversion, a little later in this chapter.

See Also

One topic *not* addressed in depth here is the reading/writing capabilities of `Channel`s classes in the Java new I/O (NIO) package.[3] This part of NIO is more complex to use than either `Files` or the input/output streams, and the benefits accrue primarily in large-scale server-side processing. Recipe 4.6 provides one example of using NIO. The NIO package is given full coverage in the book *Java NIO* by Ron Hitchens (O'Reilly).

Another topic not covered here is that of having the read or write occur concurrently with other program activity. This requires the use of threads, or multiple flows of control within a single program. Threaded I/O is a necessity in many programs: those reading from slow devices such as tape drives, those reading from or writing to network connections, and those with a GUI. For this reason, the topic is given considerable attention, in the context of multithreaded applications, in Chapter 11.

For traditional I/O topics, Elliotte Rusty Harold's *Java I/O*, although somewhat dated, should be considered the antepenultimate documentation. The penultimate reference is the Javadoc documentation, while the ultimate reference is, if you really need it, the source code for the Java API. Due in part to the quality of the Javadoc documentation, I have not needed to refer to the source code in writing this chapter.

10.6 Reading and Writing Files

Problem

The Java documentation doesn't have methods for opening files. How do I open a text file and then either read it a line at a time, or get a collection of all the lines?

Solution

Use the `Files::lines()` method, which returns a `Stream` of `String`s. Or, use `Files.readAllLines()`, which returns a `List<String>`. Or, use `Files.newBufferedReader()`, `Files.newBufferedWriter()`, `Files.newInputStream()`, and `Files.newOutputStream()`. Or, construct a `FileReader`, a `FileInputStream`, or a `BufferedReader`, and use the older `while ((line == readLine()) != null)` pattern.

Discussion

There is no explicit open operation,[4] perhaps as a kind of rhetorical flourish of the Java API's object-oriented design.

The quickest way to process a text file a line at a time is to use `Files.lines()`, which takes a `Path` argument and returns a functional `Stream<String>` into which it feeds the lines from the file:

```
Files.lines(Path.of("myFile.txt")).forEach(System.out::println);
```

The `Files` class has several other static methods that open a file and read some or all of it:

`List<String> Files.readAllLines(Path)`

Reads the whole file into a `List<String>`

`String Files.readString(Path)`

Returns the whole file as a multi-line string

`byte[] Files.readAllBytes`

Reads the whole file into an array of bytes

There is a series of methods with names like `newReader()`, `newBufferedWriter()`, etc., each of which takes a `Path` argument and returns the appropriate `Reader/Writer` or `InputStream/OutputStream`. As discussed in Recipe 10.2, a `Path` is a descriptor for an abstract path (filename) that may or may not exist. The explicit constructors for a `FileReader`, `FileWriter`, `FileInputStream`, or `FileOutputStream` take a filename or an instance of the older `File` class containing the path. These operations correspond to the "open" operation in most other languages' I/O packages.

Historically, Java used to require use of the code pattern `while ((line == readLine()) != null)` to read lines from a `BufferedReader`. This still works, of course, and will continue to work until the last JavaBean sets in the west, in the far distant future.

Example 10-7 shows the code for each of these ways to read lines from a file.

Example 10-7. main/src/main/java/io/ReadLines.java (reading lines from a file)

```
System.out.println("Using Path.lines()");
Files.lines(Path.of(fileName)).forEach(System.out::println);

System.out.println("Using Path.readAllLines()");
List<String> lines = Files.readAllLines(Path.of(fileName));
lines.forEach(System.out::println);

System.out.println("Using BufferedReader.lines().forEach()");
new BufferedReader(new FileReader(fileName)).lines().forEach(s -> {
  System.out.println(s);
});

System.out.println("The old-fashioned way");
BufferedReader is = new BufferedReader(new FileReader(fileName));
String line;
while ((line = is.readLine()) != null) {
  System.out.println(line);
}
```

The `BufferedReader` construction shown here illustrates what is commonly but informally called *constructor chaining*; the `FileReader` constructed by the "inner" call is passed directly to the `BufferedReader` constructor. This is also an example of the GoF Decorator pattern, where one class adds functionality—in this case, line reading—to another.

Most of these methods can throw the checked exception `IOException`, so you must have a `throws` clause or a `try/catch` around these invocations.

If you create an `InputStream`, `OutputStream`, `Reader`, or `Writer`, you should close it when finished. This avoids memory leaks and, in the case of writing, ensures that all buffered data is actually written to disk. One way to ensure this is not forgotten is to use the `try-with-resources` syntax. This puts the declaration and definition of a `Closeable` resource into the `try` statement:

```
static void oldWayShorter() throws IOException {
  try (BufferedReader is =
    new BufferedReader(new FileReader(INPUT_FILE_NAME));
    BufferedOutputStream bytesOut = new BufferedOutputStream(
      new FileOutputStream(OUTPUT_FILE_NAME.replace("\\.", "-1.")));) {
```

```
    // Read from is, write to bytesOut
    String line;
    while ((line = is.readLine()) != null) {
      line = doSomeProcessingOn(line);
      bytesOut.write(line.getBytes("UTF-8"));
      bytesOut.write('\n');
    }

  }
}
```

The `lines()` and read-related methods in `Files` obviate the need for closing the resource, but not the need for handling `IOException`; the compiler or IDE will remind you if you forget those.

There are options that can be passed to the `Files` methods that open a file; these are discussed in the sidebar "Understanding I/O Options: StandardOpenOptions, FileAttribute, PosixFileAttribute, and More".

To read the entire contents of a file into a single string in Java 8+, use `Files.readString()`:

```
String input = Files.readString(Path.of(INPUT_FILE_NAME)));
```

In older Java versions, use my `FileIO.readerToString()` method. Either of these will read the entire named file into one long string, with embedded newline (`\n`) characters between each line. To read a binary file, use `Files.readAllBytes()` instead.

When you need to read or write binary data, as opposed to text, just use a `DataInputStream` or `DataOutputStream`.

The `Stream` classes have been in Java since the beginning of time and are optimal for reading and writing bytes rather than characters. The data layer over them, comprising `DataInputStream` and `DataOutputStream`, is configured for reading and writing binary values, including all of Java's built-in types. Suppose that you want to write a binary integer plus a binary floating-point value into a file and read it back later. This code shows the writing part:

```
public class WriteBinary {
  public static void main(String[] argv) throws IOException {
```

```
    int i = 42;
    double d = Math.PI;
    String FILENAME = "binary.dat";
    DataOutputStream os = new DataOutputStream(
      new FileOutputStream(FILENAME));
    os.writeInt(i);
    os.writeDouble(d);
    os.close();
    System.out.println("Wrote " + i + ", " + d + " to file " + FILENAME);
  }
}
```

Should you need to write all the fields from an object, you should probably use one of the methods described in Recipe 14.6.

Formatted Printing

Should you want an easy way to use the `java.util.Formatter` class's capability for simple printing tasks, use the `printf()` methods in `PrintStream/PrintWriter`. The `Formatter` class is patterned after C's `printf` routines and has been discussed in Recipe 3.2.

10.7 Scanning Input with StreamTokenizer, Scanner, Parsers

Problem

You need to scan a file with more fine-grained resolution than the `readLine()` method of the `BufferedReader` class and its subclasses.

Solution

Use a `StreamTokenizer`, `readLine()` and a `StringTokenizer`, the `Scanner` class, regular expressions (Chapter 4), or one of several third-party parser generators.

Discussion

Though you could, in theory, read a file one character at a time and analyze each character, that is a pretty low-level approach. The `read()` method in the `Reader` class is de-

fined to return `int` so that it can use the time-honored value `-1` (defined as EOF in Unix `<stdio.h>` for decades) to indicate that you have read to the end of the file:

main/src/main/java/io/ReadCharsOneAtATime.java

```
public class ReadCharsOneAtATime {

   void doFile(Reader is) throws IOException {
      int c;
      while ((c=is.read( )) != -1) {
         System.out.print((char)c);
      }
   }
}
```

Notice the cast to `char`; the program compiles fine without it, but it does not print correctly because `c` is declared as `int`. Variable `c` must be declared `int` to be able to compare against the end-of-file value `-1`. For example, the integer value corresponding to capital A treated as an `int` prints as 65, whereas with `(char)` it prints the character `A`.

We discussed the `StringTokenizer` class extensively in Recipe 3.1. The combination of `readLine()` and `StringTokenizer` provides a simple means of scanning a file. Suppose you need to read a file in which each line consists of a name like *user@host.domain*, and you want to split the lines into users and host addresses. You could use this:

```
public class ScanStringTok {

 public static void main(String[] av) throws IOException {
  if (av.length == 0)
   System.err.printf("Usage: %s filename [...]%n",
    ScanStringTok.class.getSimpleName());
  else
   for (int i=0; i<av.length; i++)
    process(av[i]);
 }

 static void process(String fileName) {
  String s = null;
  try (BufferedReader is =
    new BufferedReader(new FileReader(fileName));) {
    while ((s = is.readLine()) != null) {
     StringTokenizer st = new StringTokenizer(s, "@", true);
     String user = (String)st.nextElement();
```

```
            st.nextElement();
            String host = (String)st.nextElement();
            System.out.println("User name: " + user +
              "; host part: " + host);

            // Do something useful with the user and host parts...
          }
        } catch (NoSuchElementException ix) {
          System.err.println("Malformed input " + s);
        } catch (IOException e) {
          System.err.println(e);
        }
      }
    }
```

The `StreamTokenizer` class in java.util provides slightly more capabilities for scanning a file. It reads characters and assembles them into words, or tokens. It returns these tokens to you along with a type code describing the kind of token it found. This type code is one of four predefined types (`StringTokenizer.TT_WORD` , `TT_NUMBER` , `TT_EOF` , or `TT_EOL` for the end-of-line) or the `char` value of an ordinary character (such as 32 for the space character). Methods such as `ordinaryCharacter()` allow you to specify how to categorize characters, while others, such as `{asciidoctor-version}slashSlashComment()` , allow you to enable or disable features.

Example 10-8 shows a `StreamTokenizer` used to implement a simple immediate-mode stack-based calculator:

```
2 2 + =
4
22 7 / =
3.141592857
```

The program reads tokens as they arrive from the `StreamTokenizer` . Numbers are put on the stack. The four operators (`+` , `-` , `*` , and `/`) are immediately performed on the two elements at the top of the stack, and the result is put back on the top of the stack. The `=` operator causes the top element to be printed, but is left on the stack so that you can chain operations, as in this example:

```
4 5 * = 2 / =
20.0
10.0
```

Example 10-8. main/src/main/java/io/SimpleCalcStreamTok.java (simple calculator using `StreamTokenizer`)

```
public class SimpleCalcStreamTok {
  /** The StreamTokenizer Input */
  protected  StreamTokenizer tf;
  /** The Output File */
  protected PrintWriter out = new PrintWriter(System.out, true);
  /** The operand stack */
  protected Stack<Double> s = new Stack<>();
  /** The name of a variable */
  protected String variable;

  protected void doCalc() throws IOException {
    int iType;
    double tmp;

    while ((iType = tf.nextToken()) != StreamTokenizer.TT_EOF) {
      switch(iType) {
      case StreamTokenizer.TT_NUMBER: // Found a number, push value to stack
        push(tf.nval);
        break;
      case StreamTokenizer.TT_WORD:
        // Found a variable, save its name. Not used here.
        variable = tf.sval;
        break;
      case '+':
        // + operator is commutative.
        push(pop() + pop());
        break;
      case '-':
        // - operator: order matters.
        tmp = pop();
        push(pop() - tmp);
        break;
      case '*':
        // Multiply is commutative
        push(pop() * pop());
        break;
      case '/':
        // Handle division carefully: order matters!
        tmp = pop();
        push(pop() / tmp);
        break;
      case '=':
        out.println(peek());
```

```
      break;
    default:
      out.println("What's this? iType = " + iType);
    }
  }
}
void push(double val) {
  s.push(Double.valueOf(val));
}
double pop() {
  return ((Double)s.pop()).doubleValue();
}
double peek() {
  return ((Double)s.peek()).doubleValue();
}
void clearStack() {
  s.removeAllElements();
}
```

The `Scanner` class is useful when you want to scan a text file consisting of various numbers and strings in a known format. You can read values of these types with the `Scanner`'s `next()` methods.

The `Scanner` class lets you read an input source by tokens, somewhat analogous to the `StreamTokenizer` described in Recipe 10.7. The `Scanner` is more flexible in some ways (it lets you break tokens based on spaces or regular expressions) but less flexible in others (you need to know the kind of token you are reading). This class bears some resemblance to the C-language `scanf()` function, but in the `Scanner` you specify the input token types by calling methods like `nextInt()`, `nextDouble()`, and so on. Here is a simple example of scanning:

```
String sampleDate = "25 Dec 1988";

try (Scanner sDate = new Scanner(sampleDate)) {
  int dayOfMonth = sDate.nextInt();
  String month = sDate.next();
  int year = sDate.nextInt();
  System.out.printf("%d-%s-%02d%n", year, month, dayOfMonth);
}
```

The `Scanner` recognizes Java's eight built-in types, in addition to `BigInteger` and `BigDecimal`. It can also return input tokens as `String`s or by matching regular expres-

sions (see Chapter 4). Table 10-7 lists the "next" methods and corresponding "has" methods; the "has" method returns true if the corresponding "next" method would succeed. There is no `nextString()` method; either use `nextLine()` to read an entire line as a string, or use `next()` to get the next token as a `String`.

Table 10-7. Scanner methods

Returned type	has **method**	next **method**	**Comment**
`String`	`hasNext()`	`next()`	The next complete token from this scanner
String	hasNext(Pattern)	next(Pattern)	The next string that matches the given regular expression (regex)
`String`	`hasNext(String)`	`next(String)`	The next token that matches the regex pattern constructed from the specified string
BigDecimal	hasNextBigDecimal()	nextBigDecimal()	The next token of the input as a BigDecimal
`BigInteger`	`hasNextBigInteger()`	`nextBigInteger()`	The next token of the input as a `BigInteger`
boolean	hasNextBoolean()	nextBoolean()	The next token of the input as a boolean
`byte`	`hasNextByte()`	`nextByte()`	The next token of the input as a `byte`
double	hasNextDouble()	nextDouble()	The next token of the input as a double
`float`	`hasNextFloat()`	`nextFloat()`	The next token of the input as a `float`
int	hasNextInt()	nextInt()	The next token of the input as an int
`String`	N/A	`nextLine()`	Reads up to the end-of-line, including the line ending
long	hasNextLong()	nextLong()	The next token of the input as a long
`short`	`hasNextShort()`	`nextShort()`	The next token of the input as a `short`

The `Scanner` class is constructed with an input source, which can be an `InputStream`, a `String`, or `Readable` (`Readable` is an interface that `Reader` and all its subclasses implement).

One way to use the `Scanner` class is based on the Iterator pattern, using `while (scanner.hasNext())` to control the iteration. Example 10-9 shows the simple calculator from Recipe 10.7 rewritten[5] to use the `Scanner` class.

Example 10-9. main/src/main/java/io/simpleCalcScanner.java (simple calculator using `java.util.Scanner`)

```
public class SimpleCalcScanner {
 /** The Scanner */
 protected Scanner scan;

 /** The output */
 protected PrintWriter out = new PrintWriter(System.out, true);

 /** The variable name (not used in this version) */
 protected String variable;

 /** The operand stack; no operators are pushed, so it can be a stack of Double */
 protected Stack<Double> s = new Stack<>();

 /* Driver - main program */
 public static void main(String[] args) throws IOException {
  if (args.length == 0)
   new SimpleCalcScanner(
    new InputStreamReader(System.in)).doCalc();
  else
   for (String arg : args) {
    new SimpleCalcScanner(arg).doCalc();
   }
 }

 /** Construct a SimpleCalcScanner by name */
 public SimpleCalcScanner(String fileName) throws IOException {
  this(new FileReader(fileName));
 }

 /** Construct a SimpleCalcScanner from an open Reader */
 public SimpleCalcScanner(Reader rdr) throws IOException {
  scan = new Scanner(rdr);
 }
```

```
/** Construct a SimpleCalcScanner from a Reader and a PrintWriter */
public SimpleCalcScanner(Reader rdr, PrintWriter pw) throws IOException {
 this(rdr);
 setWriter(pw);
}

/** Change the output to go to a new PrintWriter */
public void setWriter(PrintWriter pw) {
 out = pw;
}

protected void doCalc() throws IOException {
 double tmp;

 while (scan.hasNext()) {
  if (scan.hasNextDouble()) {
   push(scan.nextDouble());
  } else {
   String token;
   switch(token = scan.next()) {
   case "+":
    // Found + operator, perform it immediately.
    push(pop() + pop());
    break;
   case "-":
    // Found - operator, perform it (order matters).
    tmp = pop();
    push(pop() - tmp);
    break;
   case "*":
    // Multiply is commutative
    push(pop() * pop());
    break;
   case "/":
    // Handle division carefully: order matters!
    tmp = pop();
    push(pop() / tmp);
    break;
   case "=":
    out.println(peek());
    break;
   default:
    out.println("What's this? " + token);
    break;
   }
  }
```

```
    }
  }

  void push(double val) {
    s.push(Double.valueOf(val));
  }

  double pop() {
    return ((Double)s.pop()).doubleValue();
  }

  double peek() {
    return ((Double)s.peek()).doubleValue();
  }

  void clearStack() {
    s.removeAllElements();
  }
}
```

Although the `StreamTokenizer` class (see Recipe 10.7) and `Scanner` (see Recipe 10.7) are useful, they know only a limited number of tokens and have no way of specifying that the tokens must appear in a particular order. To do more advanced scanning, particularly if you need to parse a file whose structure can be described as grammatical (in the sense of computer languages, not natural languages), you should consider using a tool known as a *parser generator*.

Parser generators have a long history in computer science. The best-known examples are the C-language `yacc` (Yet Another Compiler Compiler) and `lex`, released with Seventh Edition Unix in the 1970s and discussed in *lex & yacc* by Doug Brown et al. (O'Reilly), and their open source clones *bison* and *flex*. These tools let you specify the lexical structure of your input using some pattern language such as regular expressions (see Chapter 4). For example, you might say that an email address consists of a series of alphanumerics, followed by an at sign (@), followed by a series of alphanumerics with periods embedded, like this:

```
email:  [A-Za-z0-9]+@[A-Za-z0-0.]+
or
email: \w+@[\w.]+
```

The tool then writes code that recognizes the characters you have described. These tools also have a grammatical specification, which says, for example, that the keyword `EMAIL` must appear, followed by a colon, followed by a `name` token, as previously defined.

There are several good third-party parser generator tools for Java. They vary widely based on complexity, power, and ease of use:

- One of the best known and most elaborate is ANTLR.
- JavaCC is an open source project.
- JParsec lets you write the parser in straight Java, so it's all built at compile time (most of the others require a separate parse generation step, with the build and debugging issues that raises). JParsec is on GitHub.
- JFlex and CUP work together like the original *yacc* and *lex*, as grammar parser and lexical scanner, respectively.
- Parboiled uses *Parsing Expression Grammar* (PEG) to also build the parser at compile time. See GitHub for more information.
- The *Rats!* parser generator was part of the eXTensible Compiler Project at New York University. (The project's website is no longer available.)
- There are others—a more complete list was maintained at Java Source; it is still online but is not being updated.

These parser generators can be used to write grammars for a wide variety of programs, from simple calculators—such as the one in Recipe 10.7—through HTML and CORBA/IDL, up to full Java and C/C++ parsers. Examples of these are included with the downloads. Unfortunately, the learning curve for parsers in general precludes providing a simple and comprehensive example here, let alone comparing them intelligently. Refer to the documentation and the numerous examples provided with each distribution.

As an alternative to using one of these parser generators, you could simply roll your own recursive descent parser; and once you learn how to do so, you may find it's not really that difficult, quite possibly even less hassle than dealing with the extra parser generator software (depending on the complexity of the grammar involved, obviously).

Java developers have a range of choices, including simple line-at-a-time scanners using `StringTokenizer`, fancier token-based scanners using `StreamTokenizer`, a `Scanner` class to scan simple tokens (see earlier in Recipe 10.7), regular expressions (see Chapter 4), and third-party solutions including grammar-based scanners based on the parsing tools listed here.

10.8 Reading from the Standard Input or from the Console/Controlling Terminal

Problem

You want to read from the program's standard input or directly from the program's controlling terminal or console terminal.

Solution

For the standard input, read bytes by wrapping a `BufferedInputStream()` around `System.in`. For reading text, use an `InputStreamReader` and a `BufferedReader`. For the console or controlling terminal, use Java's `System.console()` method to obtain a `Console` object, and use its methods.

Discussion

Sometimes you really do need to read from the standard input, or console. One reason for this is that simple test programs are often console-driven. Another is that some programs naturally require a lot of interaction with the user and you want something faster than a GUI (consider an interactive mathematics or statistical exploration program). Yet another is piping the output of one program directly to the input of another, a very common operation among Unix users and quite valuable on other platforms, such as Windows, that support this operation.

Standard input and output streams

All desktop operating systems support the notion of *standard input* (a keyboard, a file, or the output from another program) and *standard output* (a terminal window, a printer, a file on disk, or the input to yet another program).[6] Most such systems also support a standard error output so that error messages can be seen by the user even if the standard output is being redirected. When programs on these platforms are started up, the system assigns the three streams to particular platform-dependent handles, or *file descriptors*. The net result is that ordinary programs on these operating systems can read the standard input or write to the standard output or standard error stream without having to open any files or make any other special arrangements.

Java continues this tradition and enshrines it in the `System` class. The static variables `System.in`, `System.out`, and `System.err` are connected to the three operating system streams before your program begins execution (an application is free to reassign these; see Recipe 10.11). So, to read the standard input, you need only refer to the variable `System.in` and call its methods. For example, to read one byte from the standard input, you call the read method of `System.in`, which returns the byte in an `int` variable:

```
int b = System.in.read( );
```

But is that enough? No, because the `read()` method can throw an `IOException`. So you must either declare that your program throws an `IOException`:

```
public static void main(String args[]) throws IOException {
...
}
```

or you can put a `try` / `catch` block around the `read()` method:

```
int b = 0;
try {
  b = System.in.read();
  System.out.println("Read this data: " + (char)b);
} catch (Exception e) {
  System.out.println("Caught " + e);
}
```

In this case, it makes sense to print the results inside the `try` block because there's no point in trying to print the value you read, if the `read()` threw an `IOException`.

That code works and gives you the ability to read a byte at a time from the standard input. But most applications are designed in terms of larger units, such as integers, or lines of text. To read a value of a known type, such as `int` or `String`, from the standard input, you can use the `Scanner` class (covered in more detail earlier in Recipe 10.7):

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
String s = sc.nextLine();
```

But this is fairly limiting. To read characters of text with an input character converter so that your program will work with multiple input encodings around the world, use a `Reader` class. The particular subclass that allows you to read lines of characters is a `BufferedReader`. But there's a hitch. Remember I mentioned those two categories of input classes, `Stream`s and `Reader`s? I also said that `System.in` is a `Stream`, and you want a `Reader`. How do you get from a `Stream` to a `Reader`? A crossover class called `InputStreamReader` is tailor-made for this purpose. Just pass your `Stream` (like `System.in`) to the `InputStreamReader` constructor and you get back a `Reader`, which you in turn pass to the `BufferedReader` constructor. The usual idiom for writing this in Java is to nest the constructor calls:

```
BufferedReader is = new BufferedReader(new InputStreamReader(System.in));
```

You can then read lines of text using the `readLine()` method. This method takes no argument and returns a `String` that is made up for you by `readLine()` containing the characters (converted to Unicode) from the next line of text in the file (the newline character(s) are not included). When there are no more lines of text, the literal value `null` is returned:

```
public class CatStdin {

    public static void main(String[] av) {
        try (BufferedReader is =
                new BufferedReader(new InputStreamReader(System.in))) {
            String inputLine;

            while ((inputLine = is.readLine()) != null) {
                System.out.println(inputLine);
            }
        } catch (IOException e) {
            System.out.println("IOException: " + e);
        }
    }
}
```

To read a single `Integer` from the standard input, either use a `Scanner` or read a line and parse it using `Integer.parseInt()`. To read a series of integers, one per line, you could combine these with a functional style, since the `BufferedReader` has a `lines()` method that produces a `Stream<String>`:

```
public class ReadStdinIntsFunctional {
  private static Stream<Integer> parseIntSafe(String s) {
    try {
      return Stream.of(Integer.parseInt(s));
    } catch (NumberFormatException e) {
      return Stream.empty();
    }
  }

  public static void main(String[] args) throws IOException {
    try (BufferedReader is =
        new BufferedReader(new InputStreamReader(System.in));) {
      is.lines()
        .flatMap(ReadStdinIntsFunctional::parseIntSafe)
        .forEach(System.out::println);
    }
  }
}
```

The Console (Controlling Terminal)

The `Console` class is intended for reading directly from a program's controlling terminal. When you run an application from a *terminal window* or *command prompt window* on most systems, its console and its standard input are both connected to the terminal, by default. However, the standard input can be changed by piping or redirection on most OSes. If you really want to read from wherever the user is sitting, bypassing any indirections, then the `Console` class is usually your friend.

You cannot instantiate `Console` yourself; you must get an instance from the `System` class's `console()` method. You can then call methods such as `readLine()`, which behaves largely like the method of the same name in the `BufferedReader` class used in the previous recipe.

The following code shows an example of prompting for a name and reading it from the console:

main/src/main/java/io/ConsoleRead.java

```
public class ConsoleRead {
  public static void main(String[] args) {
    String name = System.console().readLine("What is your name?");
    System.out.println("Hello, " + name.toUpperCase());
```

```
  }
}
```

One complication is that the `System.console()` method can return `null` if the console isn't connected. So production-quality code should always check for `null` before trying to use `Console`.

The `Console` class is quite useful for reading a password without having it echo. This has been a standard facility of command-line applications for decades, as the most obvious way of preventing *shoulder surfing*—somebody looking over your shoulder to see your password. No-echo password reading works in Java: the `Console` class has a `readPassword()` method that takes a `prompt` argument, intended to be used like: `cons.readPassword("Password:")`. This method returns an array of bytes, which can be used directly in some encryption and security APIs, or can easily be converted into a `String`. It is generally advised to overwrite the byte array after use to prevent security leaks when other code can access the stack, although the benefits of this are probably reduced when you've constructed a `String`. There's an example of this in the online code in *io/ReadPassword.java*.

10.9 Copying a File

Problem

You need to copy a file in its entirety.

Solution

Use one of the Java 11 `Files.copy()` methods. If on an older release, use the explicit read and write methods in the `Reader`s/`Writer`s or `InputStream`s/`OutputStream`s.

Discussion

The `Files` class has several overloads of a `copy` method that makes quick work of this requirement:

```
long copy(InputStream src, Path dest, CopyOption...) throws IOException;
long copy(Path src, OutputStream dest) throws IOException;
```

```
// Returns the Path to the target:
Path copy(Path src, Path dest, CopyOption...) throws java.io.IOException;
```

For example:

```
Path p = Paths.get("my_new_file");
InputStream is = // open some file for reading
long newFileSize = Files.copy(is, p);
```

Long ago, Java's I/O facilities did not package a lot of the common operations like copying one file to another or reading a file into a `String`. So back then I wrote my own package of helper methods. Users of older JDK versions may want to use `FileIO` from my utilities package com.darwinsys.util. Here's a simple demo program that uses `FileIO` to copy a source file to a backup file:

main/src/demo/java/io/FileIoDemo.java

```
package com.darwinsys.io;

import java.io.IOException;

public class FileIoDemo {
   public static void main(String[] av) {
      try {
         FileIO.copyFile("FileIO.java", "FileIO.bak");
         FileIO.copyFile("FileIO.class", "FileIO-class.bak");
      } catch (IOException e) {
         System.err.println(e);
      }
   }
}
```

My `copyFile` method takes several forms, depending on whether you have two filenames, a filename and a `PrintWriter`, and so on. The code for `FileIO` itself is not shown here but is online, in the *darwinsys-api* download.

10.10 Reassigning the Standard Streams

Problem

You need to reassign one or more of the standard streams `System.in`, `System.out`, or `System.err`.

Solution

Construct an `InputStream` or `PrintStream` as appropriate, and pass it to the appropriate set method in the `System` class.

Discussion

The ability to reassign these streams corresponds to what Unix (or DOS command line) users think of as *redirection*, or *piping*. This mechanism is commonly used to make a program read from or write to a file without having to explicitly open it and go through every line of code changing the read, write, print, etc. calls to refer to a different stream object. The open operation is performed by the command-line interpreter in Unix or DOS or by the calling class in Java.

Although you could just assign a new `PrintStream` to the variable `System.out`, best practice is to use the defined method to replace it:

```
String LOGFILENAME = "error.log";
System.setErr(new PrintStream(new FileOutputStream(LOGFILENAME)));
System.out.println("Please look for errors in " + LOGFILENAME);
// Now assume this is somebody else's code; you'll see it
//   writing to stderr...
int[] a = new int[5];
a[10] = 0;  // here comes an ArrayIndexOutOfBoundsException
```

The stream you use can be one that you've opened, as here, or one you inherited:

```
System.setErr(System.out);    // merge stderr and stdout to same output stream.
```

It could also be a stream connected to or from another `Process` you've started (see Recipe 18.1), a network socket, or a URL. Anything that gives you a stream can be used.

10.11 Duplicating a Stream as It Is Written

Problem

You want anything written to a stream, such as the standard output `System.out` or the standard error `System.err`, to appear there but *also* be logged in to a file.

Solution

Subclass `PrintStream` and have its `write()` methods write to two streams. Then use `system.setErr()` or `setOut()` to replace the existing standard stream with a `PrintStream` subclass.

Discussion

Some classes are meant to be subclassed. Here we're just subclassing `PrintStream` and adding a bit of functionality: a second `PrintStream`! I wrote a class called `TeePrintStream`, named after the ancient Unix command `tee`. That command allowed you to duplicate, or "tee off" (a term derived from a plumber's pipe tee, not from golf or the local pest), a copy of the data being written on a pipeline between two programs.

The original Unix `tee` command is used like this: the `|` character creates a pipeline in which the standard output of one program becomes the standard input to the next. This often-used example of pipes shows how many users are logged in to a Unix server:

```
who | wc -l
```

This runs the `who` program (which lists who is logged in to the system, one name per line, along with the terminal port and login time) and sends its output, not to the terminal, but rather into the standard input of the word count (`wc`) program. Here, `wc` is being asked to count lines, not words, hence the `-l` option. To `tee` a copy of the intermediate data into a file, you might say:

```
who | tee wholist | wc -l
```

which creates a file *wholist* containing the data. For the curious, the file *wholist* might look something like this:

```
ian     ttyC0    Mar 14 09:59
ben     ttyC3    Mar 14 10:23
ian     ttyp4    Mar 14 13:46  (laptop.darwinsys.com)
```

So both the previous command sequences would print `3` as their output.

`TeePrintStream` is my attempt to capture the spirit of the `tee` command. It can be used like this:

```
System.setErr(new TeePrintStream(System.err, "err.log"));
// ...lots of code that occasionally writes to System.err... Or might.
```

`System.setErr()` is a means of specifying the destination of text printed to `System.err` (there are also `System.setOut()` and `System.setIn()`). This code results in any messages that printed to `System.err` printing to wherever `System.err` was previously directed (normally the terminal, but possibly a text window in an IDE) and to the file *err.log*.

This technique is not limited to the three standard streams. A `TeePrintStream` can be passed to any method that wants a `PrintStream`, or, for that matter, an `OutputStream`. And you can adapt the technique for `BufferedInputStreams`, `PrintWriters`, `BufferedReaders`, and so on.

Example 10-10 shows the source code for `TeePrintStream`.

Example 10-10. main/src/main/java/io/TeePrintStream.java

```
public class TeePrintStream extends PrintStream {
 /** The original/direct print stream */
 protected PrintStream parent;

 /** The filename we are tee-ing too, if known;
  * intended for use in future error reporting.
  */
 protected String fileName;

 /** The name for when the input filename is not known */
 private static final String UNKNOWN_NAME = "(opened Stream)";

 /** Construct a TeePrintStream given an existing PrintStream,
  * an opened OutputStream, and a boolean to control auto-flush.
  * This is the main constructor, to which others delegate via "this".
```

```
 */
public TeePrintStream(PrintStream orig, OutputStream os, boolean flush)
throws IOException {
  super(os, true);
  fileName = UNKNOWN_NAME;
  parent = orig;
}

/** Construct a TeePrintStream given an existing PrintStream and
 * an opened OutputStream.
 */
public TeePrintStream(PrintStream orig, OutputStream os)
throws IOException {
  this(orig, os, true);
}

/* Construct a TeePrintStream given an existing Stream and a filename.
 */
public TeePrintStream(PrintStream os, String fn) throws IOException {
  this(os, fn, true);
}

/* Construct a TeePrintStream given an existing Stream, a filename,
 * and a boolean to control the flush operation.
 */
public TeePrintStream(PrintStream orig, String fn, boolean flush)
throws IOException {
  this(orig, new FileOutputStream(fn), flush);
  fileName = fn;
}

/** Return true if either stream has an error. */
public boolean checkError() {
  return parent.checkError() || super.checkError();
}

/** override write(). This is the actual "tee" operation. */
public void write(int x) {
  parent.write(x);  // "write once;
  super.write(x);    // write somewhere else."
}

/** override write(). This is the actual "tee" operation. */
public void write(byte[] x, int o, int l) {
  parent.write(x, o, l);  // "write once;
  super.write(x, o, l);  // write somewhere else."
```

```
    }

    /** Close both streams. */
    public void close() {
      parent.close();
      super.close();
    }

    /** Flush both streams. */
    public void flush() {
      parent.flush();
      super.flush();
    }
  }
```

It's worth mentioning that I do *not* need to override all the polymorphic forms of `print()` and `println()`. Because these all ultimately use one of the forms of `write()`, if you override the `print` and `println` methods to do the `tee`-ing as well, you can get several additional copies of the data written out.

10.12 Reading/Writing a Different Character Set

Problem

You need to read or write a text file using a particular encoding.

Solution

Convert the text to or from internal Unicode by specifying a converter when you construct an `InputStreamReader` or `PrintWriter`.

Discussion

Classes `InputStreamReader` and `OutputStreamWriter` are the bridge from byte-oriented `Stream`s to character-based `Reader`s. These classes read or write bytes and translate them to or from characters according to a specified character encoding. The UTF-16 character set used inside Java (`char` and `String` types) is a 16-bit character set. But most character sets—such as ASCII, Swedish, Spanish, Greek, Turkish, and many oth-

ers—use only a small subset of that. In fact, many European language character sets fit nicely into 8-bit characters. Even the larger character sets (script-based and pictographic languages) don't all use the same bit values for each particular character. The encoding, then, is a mapping between Java characters and an external storage format for characters drawn from a particular national or linguistic character set.

To simplify matters, the `InputStreamReader` and `OutputStreamWriter` constructors are the only places where you can specify the name of an encoding to be used in this translation. If you do not specify an encoding, the platform's (or user's) default encoding is used. `PrintWriter`s, `BufferedReader`s, and the like all use whatever encoding the `InputStreamReader` or `OutputStreamWriter` class uses. Because these bridge classes only accept `Stream` arguments in their constructors, the implication is that if you want to specify a nondefault converter to read or write a file on disk, you must start by constructing not a `FileReader` or `FileWriter`, but a `FileInputStream` or `FileOutputStream`:

```
// io/UseConverters.java
BufferedReader fromKanji = new BufferedReader(
   new InputStreamReader(new FileInputStream("kanji.txt"), "EUC_JP"));
PrintWriter toSwedish = new PrinterWriter(
   new OutputStreamWriter(new FileOutputStream("sverige.txt"), "Cp278"));
```

Not that it would necessarily make sense to read a single file from Kanji and output it in a Swedish encoding. For one thing, most fonts would not have all the characters of both character sets; and, at any rate, the Swedish encoding certainly has far fewer characters in it than the Kanji encoding. A list of the supported encodings is also in the JDK documentation, in the file *docs/guide/internat/encoding.doc.html*. A more detailed description is found in Appendix B of *Java I/O*.

10.13 Those Pesky End-of-Line Characters

Problem

You really want to know about end-of-line characters, particulary for cross-platform file exchange.

Solution

Use `\r` and `\n` in whatever combination makes sense.

Discussion

If you are reading text (or bytes containing ASCII characters) in line mode using the `readLine()` method, you'll never see the end-of-line characters, and the same applies if you're using a `PrintWriter` with its `println()` method. Thus you won't be cursed with having to figure out whether `\n`, `\r`, or `\r\n` appears at the end of each line.

If you want that level of detail, you have to read the characters or bytes one at a time, using the `read()` methods. The only time I've found this necessary is in networking code, where some of the line-mode protocols assume that the line ending is `\r\n`. Even here, though, you can still work in line mode. When writing, pass `\r\n` into the `print()` (not dealing with the characters):

```
outputSocket.print("HELO " + myName + "\r\n");
String response = inputSocket.readLine();
```

For the curious, the strange spelling of "hello" is used in SMTP, the mail-sending protocol, where commands are four letters.

10.14 Beware Platform-Dependent File Code

Problem

Chastened by the previous recipe, you now wish to write only platform-independent code.

Solution

Use `readLine()` and `println()`. Avoid use of `\n` by itself.

Discussion

As mentioned in Recipe 10.13, if you just use `readLine()` and `println()`, you won't have to think about the line endings. But a particular problem, especially for former programmers of C and related languages, is using the `\n` character in text strings to mean a newline. What is particularly distressing about this code is that it works—sometimes—usually on the developer's own platform. But it will probably fail someday, on some other system:

```
String myName;
public static void main(String[] argv) {
 BadNewline jack = new BadNewline("Jack Adolphus Schmidt, III");
 System.out.println(jack);
}
/**
 * DON'T DO THIS. THIS IS BAD CODE.
 */
public String toString() {
 return super.toString() + "\n" + myName;
}

// The obvious Constructor is not shown for brevity; it's in the code
```

The real problem is not that it fails on some platforms, though. What's really wrong is that it mixes formatting and I/O, or tries to. Don't mix line-based display with `toString()`; avoid *multiline strings*—output from `toString()` or any other string-returning method. If you need to write multiple strings, then say what you mean:

```
String myName;
public static void main(String[] argv) {
 GoodNewline jack = new GoodNewline("Jack Adolphus Schmidt, III");
 jack.print(System.out);
}

protected void print(PrintStream out) {
 out.println(toString());  // classname and hashcode
 out.println(myName);    // print name  on next line
}
```

Alternatively, if you need multiple lines, you could return an array or `List` of strings.

10.15 Reading and Writing JAR or ZIP Archives

Problem

You need to create and/or extract from a JAR archive or a file in the well-known ZIP archive format, as established by PKZIP and used by Unix zip/unzip and WinZip.

Solution

You could use the `jar` program in the JDK because its file format is identical to the ZIP format, with the addition of a *META-INF* directory for additional metadata. But because this is a book about programming, you are probably more interested in the `ZipFile` and `ZipEntry` classes and the stream classes to which they provide access.

Discussion

The class `java.util.zip.ZipFile` allows you to read the contents of a JAR or ZIP-format file. When constructed, it creates a series of `ZipEntry` objects, one to represent each entry in the archive. In other words, the `ZipFile` represents the entire archive, and the `ZipEntry` represents one entry, or one file that has been stored (and compressed) in the archive. The `ZipEntry` has methods like `getName()`, which returns the name that the file had before it was put into the archive, and `getInputStream()`, which gives you an `InputStream` that will transparently uncompress the archive entry by filtering it as you read it. To create a `ZipFile` object, you need either the name of the archive file or a `File` object representing it:

```
ZipFile zippy = new ZipFile(fileName);
```

To see whether a given file is present in the archive, you can call the `getEntry()` method with a filename. More commonly, you'll want to process all the entries; for this, use the `ZipFile` object to get a list of the entries in the archive, in the form of an `Enumeration` (see Recipe 7.7), as is done here:

```
Enumeration all = zippy.entries( );
while (all.hasMoreElements( )) {
  ZipEntry entry = (ZipEntry)all.nextElement( );
  ...
}
```

We can also get a `Stream<JarEntry>` using the `ZipFile.stream()` method. The following one-liner will list the contents of a `ZipFile`:

```
zippy.stream().forEach(System.out::println);
```

We can then process each entry as we wish. A simple listing program could be this:

```
if (entry.isDirectory( ))
    println("Directory: " + e.getName( ));
else
    println("File: " + e.getName( ));
```

A fancier version would extract the files. The program in Example 10-11 does both: it lists the files by default, but with the `-x` (extract) switch, it actually extracts the files from the archive.

Example 10-11. main/src/main/java/io/UnZip.java

```
public class UnZip {
  /** Constants for mode listing or mode extracting */
  public static enum Mode {
    LIST,
    EXTRACT;
  }
  /** Whether we are extracting or just printing TOC */
  protected Mode mode = Mode.LIST;

  /** The ZipFile that is used to read an archive */
  protected ZipFile zippy;

  /** The buffer for reading/writing the ZipFile data */
  protected byte[] b = new byte[8092];

  /** Simple main program, construct an UnZipper, process each
   * .ZIP file from argv[] through that object.
   */
  public static void main(String[] argv) {
    UnZip u = new UnZip();

    for (int i=0; i<argv.length; i++) {
      if ("-x".equals(argv[i])) {
        u.setMode(Mode.EXTRACT);
        continue;
      }
      String candidate = argv[i];
      // System.err.println("Trying path " + candidate);
      if (candidate.endsWith(".zip") ||
        candidate.endsWith(".jar"))
          u.unZip(candidate);
      else System.err.println("Not a zip file? " + candidate);
    }
```

```
  System.err.println("All done!");
}

/** Set the Mode (list, extract). */
protected void setMode(Mode m) {
  mode = m;
}

/** Cache of paths we've mkdir()ed */
protected SortedSet<String> dirsMade;

/** For a given Zip file, process each entry. */
public void unZip(String fileName) {
  dirsMade = new TreeSet<String>();
  try {
    zippy = new ZipFile(fileName);
    @SuppressWarnings("unchecked")
    Enumeration<ZipEntry> all = (Enumeration<ZipEntry>) zippy.entries();
    while (all.hasMoreElements()) {
      getFile((ZipEntry)all.nextElement());
    }
  } catch (IOException err) {
    System.err.println("IO Error: " + err);
    return;
  }
}

protected boolean warnedMkDir = false;

/** Process one file from the zip, given its name.
 * Either print the name, or create the file on disk.
 */
protected void getFile(ZipEntry e) throws IOException {
  String zipName = e.getName();
  switch (mode) {
  case EXTRACT:
    if (zipName.startsWith("/")) {
      if (!warnedMkDir)
        System.out.println("Ignoring absolute paths");
      warnedMkDir = true;
      zipName = zipName.substring(1);
    }
    // if a directory, just return. We mkdir for every file,
    // since some widely used Zip creators don't put out
    // any directory entries, or put them in the wrong place.
    if (zipName.endsWith("/")) {
```

```
        return;
      }
      // Else must be a file; open the file for output
      // Get the directory part.
      int ix = zipName.lastIndexOf('/');
      if (ix > 0) {
        String dirName = zipName.substring(0, ix);
        if (!dirsMade.contains(dirName)) {
          File d = new File(dirName);
          // If it already exists as a dir, don't do anything.
          if (!(d.exists() && d.isDirectory())) {
            // Try to create the directory, warn if it fails.
            System.out.println("Creating Directory: " + dirName);
            if (!d.mkdirs()) {
              System.err.println(
              "Warning: unable to mkdir " + dirName);
            }
            dirsMade.add(dirName);
          }
        }
      }
      System.err.println("Creating " + zipName);
      FileOutputStream os = new FileOutputStream(zipName);
      InputStream  is = zippy.getInputStream(e);
      int n = 0;
      while ((n = is.read(b)) >0)
        os.write(b, 0, n);
      is.close();
      os.close();
      break;
    case LIST:
      // Not extracting, just list
      if (e.isDirectory()) {
        System.out.println("Directory " + zipName);
      } else {
        System.out.println("File " + zipName);
      }
      break;
    default:
      throw new IllegalStateException("mode value (" + mode + ") bad");
    }
  }
}
```

To *write* a `ZipFile`, use the `ZipOutputStream` class. This is demonstrated in Example 10-12. The program creates a new `ZipOutputStream` for the file *temp.zip*, creates two `ZipEntry` elements, and uses each of them to write some text.

Example 10-12. main/src/main/java/io/WriteZipFile.java

```
class WriteZipFile {

  public static final String FILENAME = "temp.zip";

  public static void main(String[] args) throws Exception {
    File file = new File(FILENAME);
    ZipOutputStream zf = new ZipOutputStream(new FileOutputStream(file));
    zf.putNextEntry(new ZipEntry("foo.bar.txt"));
    zf.write("Hello\n".getBytes());
    zf.putNextEntry(new ZipEntry("WriteZipFile.java"));
    Files.copy(
       Path.of("WriteZipFile.java"), zf);
    zf.closeEntry();
    zf.close();
    System.out.println("Written to " + FILENAME);
  }
}
```

Running the program creates the file, as verified by the system `unzip` command:

```
$ java src/main/java/io/WriteZipFile.java
$ unzip -t temp.zip
Archive:  temp.zip
    testing: foo.bar.txt             OK
    testing: WriteZipFile.java       OK
No errors detected in compressed data of temp.zip.
$ unzip temp.zip foo.bar.txt
Archive:  temp.zip
  inflating: foo.bar.txt
$ cat foo.bar.txt
Hello
$
```

See Also

People sometimes confuse the ZIP archive file format with the similarly named gzip compression format. Gzip-compressed files can be read or written with the `GZipInputStream` and `GZipOutputStream` classes from java.io.

10.16 Reading Files in a Filesystem-Neutral Way with getResource() and getResourceAsStream()

Problem

You want to load objects or files without referring to their absolute location in the filesystem. This is commonly used to package a file such as a configuration file into a JAR file and then access it from the code in the same file. It can also be used to provide a file (or different version of a file) in unit tests.

Solution

Use `getClass()` or `getClassLoader()` and either `getResource()` or `getResourceAsStream()`.

Discussion

There are three varieties of `getResource()` methods, some of which exist (with the exact same signature) both in the `Class` class (see Chapter 17) and in the `ClassLoader` class (see Recipe 17.9). The methods in `Class` delegate to the `ClassLoader`, so there is little difference between them. The methods are summarized in Table 10-8.

Table 10-8. The `getResource` methods

Method signature	In Class	In ClassLoader
`public InputStream getResourceAsStream(String);`	Y	Y
`public URL getResource(String);`	Y	Y
`public Enumeration<URL> getResources(String) throws IOException;`	N	Y

The first method is designed to quickly and easily locate a resource, or file, on your `CLASSPATH`. Using the `Class` version, or the other version with a standard `ClassLoader` implementation, the resource can be a physical file or a file inside a JAR file. If you define your own `ClassLoader`, your imagination is the limit, as long as it can be represented as an `InputStream`.

The common usage is shown here:

```
InputStream is = getClass().getResourceAsStream("foo.properties");
// check for null, then do something with the InputStream...
```

The second method returns a `URL`, which can be interpreted in various ways (see the discussion of reading from a URL in Recipe 14.1).

The third method, only usable with a `ClassLoader` instance, returns an `Enumeration` of `URL` objects. This is intended to return all the resources that match a given string; remember that a `CLASSPATH` can consist of an arbitrary number of directories and/or JAR files, so this method will search all of them. This is useful for finding a series of configuration files and merging them, perhaps. Or for finding out whether there is more than one resource/file of a given name on your `CLASSPATH`.

Note that the resource name can be given as either a relative path or as an absolute path. Assuming you are using Maven (see Recipe 2.4), then for the absolute path, place the file relative to the *src/main/resources/* directory. For the relative path, place the file in the same directory as your source code. The same rules apply in an IDE, assuming you have configured your IDE to treat *src/main/java* and *src/main/resources* as source folders. The idea is that resource files get copied to your `CLASSPATH` folder. For example, we have two resource files, *src/main/resources/getresourcedemo1.txt* and *src/main/java/io/myconfig.properties*, and the project is configured as described. We then execute the program shown in Example 10-13 in two different ways.

Example 10-13. main/src/main/java/io/GetResourceDemo.java

```
package io;

import java.io.*;

public class GetResourceDemo {
 private static final String MY_CONFIG_FILE = "myconfig.properties";

 public static void main(String[] args) throws Exception {
  Class<?> c = GetResourceDemo.class;

  // By file name
  System.out.println("Class is " + c.getName());
  final String pathName1 = "/getresourcedemo1.txt";
  System.out.println("pathName1 = " + pathName1);
  InputStream isOne = c.getResourceAsStream(pathName1);
  System.out.println("InputStream One = " + isOne);
  displayContents(isOne); // simple convenience routing

  // By package-relative name
  final String name = c.getPackage().getName();
  final String fullPathName = "/" + name.replace('.', '/') + "/" + MY_CONFIG_FILE;
  System.out.println("pathName2 = " + fullPathName);
  InputStream isTwo = c.getResourceAsStream(fullPathName);
  System.out.println("InputStream Two = " + isTwo);
  displayContents(isTwo);
 }
```

Build tools and IDEs will normally copy files from *src/main/resources* into their generated class folder, but they will not normally copy resource files found under *src/main/java*. Thus two invocations of the program in Example 10-13 show two different results:

```
# Show locations of the two config files
$ find . -name getresourcedemo1.txt -o -name myconfig.properties
./src/main/java/io/myconfig.properties
./src/main/resources/getresourcedemo1.txt
./target/classes/getresourcedemo1.txt
$

# Run from within an IDE
Class is io.GetResourceDemo
pathName1 = /getresourcedemo1.txt
InputStream One = java.io.BufferedInputStream@621be5d1
```

```
This is a demo file for the GetResourceDemo program.
pathName2 = /io/myconfig.properties
InputStream Two = null

# Run from command line
$ cd src/main/java
/home/ian/git/javasrc/main/src/main/java
$ java io/GetResourceDemo.java
Class is io.GetResourceDemo
pathName1 = /getresourcedemo1.txt
InputStream One = null
pathName2 = /io/myconfig.properties
InputStream Two = java.io.BufferedInputStream@2ed0fbae
This is my config file, which is my own.
$
```

WARNING

With either format, `getResource()` and `getResourceAsStream()` will return `null` if they don't find the resource; you should always check for `null` to guard against faulty deployment. If it doesn't find anything matching, `getResources()` will return an empty `Enumeration`.

If the file path has a package name, the name you pass into any of the `getResource` methods should have a slash in place of each period, reflecting how Java stores classes in subdirectories.

10.17 Creating a Transient/Temporary File

Problem

You need to create a file with a unique temporary filename and/or or arrange for a file to be deleted when your program is finished.

Solution

Use the `java.nio.file.Files` `createTempFile()` or `createTempDirectory()` method. Use one of several methods to ensure your file is deleted on exit.

Discussion

The `Files` class has static methods for creating temporary files and directories. Note that a temporary file in this context is not deleted automatically; it is simply created in a directory that is set aside for temporary files on that operating system (e.g., */tmp* on Unix). Here are the static `Files` methods for creating temporary files and directories:

`Path createTempFile(Path dir, String prefix, String suffix, FileAttribute<?> ... attrs)`

Creates a new empty file in the specified directory, using the given prefix and suffix strings to generate its name

`Path createTempFile(String prefix, String suffix, FileAttribute<?>... attrs)`

Creates an empty file in the default temporary-file directory, using the given prefix and suffix to generate its name

`Path createTempDirectory(Path dir, String prefix, FileAttribute<?>... attrs)`

Creates a new directory in the specified directory, using the given prefix to generate its name

`Path createTempDirectory(String prefix, FileAttribute<?> ... attrs)`

Creates a new directory in the default temporary-file directory, using the given prefix to generate its name

The file attributes are discussed in "Understanding I/O Options: StandardOpenOptions, FileAttribute, PosixFileAttribute, and More".

There are various ways to arrange for a file to be deleted automatically. One option is to use the legacy `java.io.File` class, which has an explicit `deleteOnExit()` method. This arranges for any file (no matter how it was created) to be deleted if it still exists when the program exits. Here we arrange for a backup copy of a program to be deleted on exit, and we also create a temporary file and arrange for it to be removed on exit. Both files are gone after the program runs:

```
public class TempFiles {
 public static void main(String[] argv) throws IOException {

   // 1. Making an existing file temporary
   // Construct a File object for the backup created by editing
   // this source file. The file probably already exists.
   // My editor creates backups by putting ~ at the end of the name.
```

```
    File bkup = new File("Rename.java~");
    // Arrange to have it deleted when the program ends.
    bkup.deleteOnExit();

    // 2. Create a new temporary file.

    // Make a file foo.tmp, in the default temp directory.
    Path tmp = Files.createTempFile("foo", "tmp");
    // Report on the filename that it made up for us.
    System.out.println("Your temp file is " + tmp.normalize());
    // Arrange for it to be deleted at exit.
    tmp.toFile().deleteOnExit();
    // Now do something with the temporary file, without having to
    // worry about deleting it later.
    writeDataInTemp(tmp);
  }

  public static void writeDataInTemp(Path tempFile) throws IOException {
    // This version is dummy. Use your imagination.
    Files.writeString(tempFile, "This is a temp file");
  }
}
```

When run on a Unix system, the program looks like this, proving that the file was created but removed when the JVM exited:

```
$ java TempFiles.java
Your temp file is /tmp/foo842332191021505468 9tmp
$ ls -l /tmp/foo842332191021505468 9tmp
ls: /tmp/foo842332191021505468 9tmp: No such file or directory
$
```

The `createTempFile()` method is like `createFile()` (see Recipe 10.3) in that it does create the file. You should also be aware that, should the JVM terminate abnormally, the deletion probably will not occur. There is no way to undo the setting of `deleteOnExit()` short of renaming the file or something drastic like powering off the computer before the program exits.

Another way to arrange for any file to be deleted when you are finished with it is to create it with the `DELETE_ON_CLOSE` option (see Table 10-5) so it will be deleted when you close the file.

A third, less likely, method is to instead use a JVM shutdown hook. `DELETE_ON_CLOSE` is probably the best option, particularly in a long-running application, like most server-side apps. In these situations, the server could be running for weeks, months, or even years. In the meantime all the temp files would accumulate and the JVM would accumulate a large list of deferred work that it needs to perform upon shutdown. You'd probably run out of disk space or server memory or some other resource. For most long-running apps of this kind, it's better to use `DELETE_ON_CLOSE` or even the explicit `delete()` operation. Another alternative is to use a scheduler service to periodically trigger removal of old temporary files.

10.18 Getting the Directory Roots

Problem

You want information about the top-level directories, such as *C:* and *D:* on Windows.

Solution

Use the static method `FileSystems.getDefault().getRootDirectories()`, which returns an `Iterable` of `Path` objects, one for each root directory. You can print them or do other operations on them.

Discussion

Operating systems differ in how they organize filesystems out of multiple disk drives or partitions. Microsoft Windows has a low-level device-oriented approach in which each disk drive has a root directory named *A:* for the first floppy drive (if you still have one!), *C:* for the first hard drive, and other letters for DVD-ROM (or CD-ROM) and network drives. This approach requires you to know the physical device on which a file is stored. Unix, Linux, and macOS have a higher-level approach with a single root directory */*, and different disks or partitions are *mounted*, or connected, into a single unified tree. This approach sometimes requires you to figure out where a device file is mounted. Perhaps neither is easier, though the Unix approach is a bit more consistent. Either way, Java makes it easy for you to get a list of the roots.

The static method `FileSystems.getDefault().getRootDirectories()` returns an `Iterable<Path>` containing the available filesystem roots for whatever platform you are running on. Here is a short program to list these:

```
FileSystems.getDefault().getRootDirectories().forEach(System.out::println);
```

```
C:> java dir_file.DirRoots
A:\
C:\
D:\
C:>
```

Run on Microsoft Windows, the program listed my floppy drive (even though the floppy drive was not only empty, but also left at home while I wrote this recipe on my notebook computer in my car in a parking lot), the hard disk drive, and the DVD-ROM drive.

On Unix there is only one root directory:

```
$ java dir_file.DirRoots
/
$
```

One thing that is left out of the list of roots is the so-called *UNC filename*. UNC filenames are used on some Microsoft platforms to refer to a network-available resource that hasn't been mounted locally on a particular drive letter. If your system still uses these, be aware they will not show up in the `listDirectoryRoots()` output.

10.19 Using the File Watcher Service to Get Notified About File Changes

Problem

You want to be notified when some other application updates one or more of the files in which you are interested.

Solution

Use the `java.nio.file.WatchService` to get notified of changes to files automatically, instead of having to examine the files periodically.

Discussion

It is fairly common for a large application to want to be notified of changes to files, without having to go and look at them periodically. For example, an IDE wants to know when files were modified by an external editor or a build script. A Java Enterprise server wants to know if the class file for components got updated. Many modern operating systems have had this capability for some time, and now it is available in Java.

These are the basic steps to using the `WatchService`:

1. Create a `Path` object representing the directory you want to watch.
2. Get a `WatchService` by calling, for example, `FileSystems.getDefault().newWatchService()`.
3. Create an array of `Kind` enumerations for the things you want to watch (in our example we watch for files being created or modified).
4. Register the `WatchService` and the `Kind` array onto the `Path` object.
5. From then on, you wait for the watcher to notify you. A typical implementation is to enter a `while (true)` loop calling the `WatchService`'s `take()` method to get an event and interpret the events to figure out what just happened.

Example 10-14 is a program that does just that. In addition, it starts another thread to actually do some filesystem operations so that you can see the `WatchService` operating.

Example 10-14. main/src/main/java/nio/FileWatchServiceDemo.java

```
public class FileWatchServiceDemo {

  final static String TEMP_DIR_PATH = "/tmp";
  static final String FILE_SEMA_FOR = "MyFileSema.for";
  final static Path SEMAPHORE_PATH = Path.of(TEMP_DIR_PATH ,FILE_SEMA_FOR);
  static volatile boolean done = false;
  final static ExecutorService threadPool = Executors.newSingleThreadExecutor();

  public static void main(String[] args) throws Throwable {
    String tempDirPath = "/tmp";
    System.out.println("Starting watcher for " + tempDirPath);
    System.out.println("Semaphore file is " + SEMAPHORE_PATH);
    Path p = Paths.get(tempDirPath);
    WatchService watcher =
      FileSystems.getDefault().newWatchService();
    Kind<?>[] watchKinds = { ENTRY_CREATE, ENTRY_MODIFY };
    p.register(watcher, watchKinds);
    threadPool.submit(new DemoService());
```

```
    while (!done) {
      WatchKey key = watcher.take();
      for (WatchEvent<?> e : key.pollEvents()) {
        System.out.println(
          "Saw event " + e.kind() + " on " +
          e.context());
        if (e.context().toString().equals(FILE_SEMA_FOR)) {
          System.out.println("Semaphore found, shutting down watcher");
          done = true;
        }
      }
      if (!key.reset()) {
        System.err.println("WatchKey failed to reset!");
      }
    }
  }

  /**
   * Nested class whose only job is to wait a while, create a file in
   * the monitored directory, and then go away.
   */
  private final static class DemoService implements Runnable {
    public void run() {
      try {
        Thread.sleep(1000);
        System.out.println("DemoService: Creating file");
        Files.deleteIfExists(SEMAPHORE_PATH); // clean up from previous run
        Files.createFile(SEMAPHORE_PATH);
        Thread.sleep(1000);
        System.out.println("DemoService: Shutting down");
      } catch (Exception e) {
        System.out.println("Caught UNEXPECTED " + e);
      }
    }
  }
}
```

10.20 Walking a File Tree (like Find)

Problem

You need to work through some or all of the files and directories in a filesystem tree, that is, recursively visit all filesystem entries under a given starting point.

Solution

Use the `walk()` or `walkFileTree()` methods in the `Files` class.

Discussion

The `Files` class has four methods for walking a file tree. Two return a lazily populated `Stream<Path>`, and the other two invoke a callback `FileVisitor` for each file or directory found. As the name hints, this is an example of the GoF Visitor pattern. My `find` implementation uses the first method; the four are summarized in Table 10-9.

Table 10-9. `Files` tree walk methods

Return	Signature
`Stream<Path>`	`walk(Path start, FileVisitOption... options)`
`Stream<Path>`	`walk(Path start, int maxDepth, FileVisitOption... options)`
`Path`	`walkFileTree(Path start, FileVisitor<? super Path> visitor)`
`Path`	`walkFileTree(Path start, Set<FileVisitOption> options, int maxDepth, FileVisitor<? super Path> visitor)`

Using the `walk()` methods is as simple as this:

```
Files.walk(startingPath).forEach(path -> {
    // Do something with Path path; might be a file, directory, or other...
}
```

That code is near the start of the `startWalkingAt()` method in my *Find.java* file (in the *javasrc* repo, in the file *main/src/main/java/dir_file/Find.java*).

The `walkFileTree()` methods both require a `FileVisitor()`, a simple interface with four callback methods, summarized in Table 10-10.

Table 10-10. `FilesVisitor` interface

Method signature	When invoked
`FileVisitResult preVisitDirectory(T dir, BasicFileAttributes attrs)`	Called before each directory is started
`FileVisitResult postVisitDirectory(T dir, IOException exc)`	Called when all the entries in a directory (including sub-directories, recursively) have been processed
`FileVisitResult visitFile(T file, BasicFileAttributes attrs)`	Called for each file in a directory
`FileVisitResult visitFileFailed(T file, IOException exc)`	Called for a file that couldn't be visited; the `IOException` will explain the failure.

Callers of these methods can implement the `FileVisitor` interface, or if only some methods are needed, subclass the `SimpleFileVisitor` class. These methods are demonstrated in Example 10-15.

Example 10-15. main/src/main/java/dir_file/WalkTreeDemo.java

```
public class WalkTreeDemo {

  public static void main(String[] args) throws IOException {
    String dir = args.length == 0 ? "." : args[0];
    Files.walkFileTree(Path.of(dir), myVisitor);
  }

  static final FileVisitor myVisitor = new FileVisitor() {
    @Override
    public FileVisitResult preVisitDirectory(Object dir,
      BasicFileAttributes attrs) throws IOException {
      System.out.println("Starting Directory " + dir);
      return FileVisitResult.CONTINUE;
    }

    @Override
    public FileVisitResult postVisitDirectory(Object dir,
      IOException exc) throws IOException {
      System.out.println("Finished Directory " + dir +
```

```
            " " + (exc != null? exc : ""));
        return FileVisitResult.CONTINUE;
      }

      @Override
      public FileVisitResult visitFile(Object file,
        BasicFileAttributes attrs) throws IOException {
        System.out.println("Visiting File " + file);
        return FileVisitResult.CONTINUE;
      }

      @Override
      public FileVisitResult visitFileFailed(Object file,
        IOException exc) throws IOException {
        System.out.println(
          MessageFormat.format("FAILURE visiting {0} ({1})", file, exc));
        return FileVisitResult.CONTINUE;
      }
    };
  }
```

My simplified Find program implements a subset of the Windows Find Files dialog or the Unix `find` command. It has most of the structure needed to build a more complete version of either of these. The program accepts the following options from the standard Unix `find` (with limits):

`-n name`

Name to look for. Can include shell wildcards if quoted from the shell.

`-s size`

Size of file to look for. Can prefix with a plus sign to indicate greater than or a minus sign to indicate less than.

`-a`, *`-o`*

And or *or*, but currently only one of these, between an `-n` and an `-s`.

The entire Find program is too long to include in the book, but can be found in the *javasrc* repo, in the files *main/src/main/java/dir_file* directory. The files are also available at *https://github.com/IanDarwin/javasrc/blob/main/main/src/main/java/dir_file*.

[1] A `Paths` object exists and has two methods that return `Path` objects, but this class is not recommended and is probably going to be deprecated soon.

[2] The Institute of Electrical and Electronics Engineers, a nonprofit standards organization.

[3] A poor choice of name: it was new in Java SE 1.4. But it is newer than `InputStream` / `OutputStream` (Java 1.0) and `Reader`s/ `Writer`s (1.1).

[4] Not strictly true; there is, but only in the `java.nio.FileChannel` class, which we're not covering.

[5] If this were code in a maintained project, I might factor out some of the common code among these two calculators, as well as the one in Recipe 5.13, and divide the code better using interfaces. However, this would detract from the simplicity of self-contained examples.

[6] Even IBM's MVT and MVS operating systems provided `SYSIN` and `SYSOUT` in their batch Job Control Language.

Previous chapter
9. Functional Programming Techniques: Functional Interfaces, Streams, and Parallel Collections

Next chapter
11. Threaded Java